

Convolutional Neural Networks

Iftekharul Islam

Contents



Background



Building Blocks



Case Studies



Applications

Background

Fully Connected Neural Networks (FCNNs)

- Input layer $\rightarrow$ hidden layer(s) $\rightarrow$ output layer
- **Universal function approximators.** Can learn complex, non-linear relationships in data.
- Effective for various tasks where data can be represented as a flat feature vector.
 - **Example:** Spam Detection, Credit Scoring, Medical Diagnosis etc.

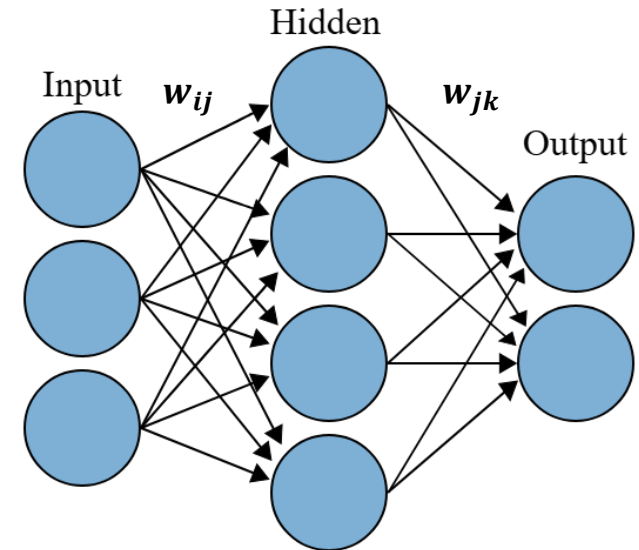
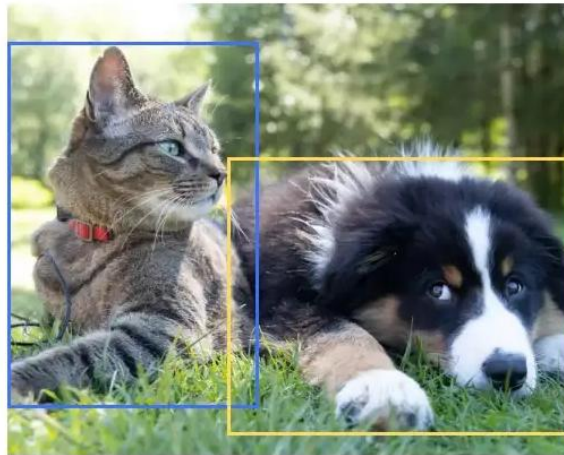


Image-based Tasks



Classification

Cat



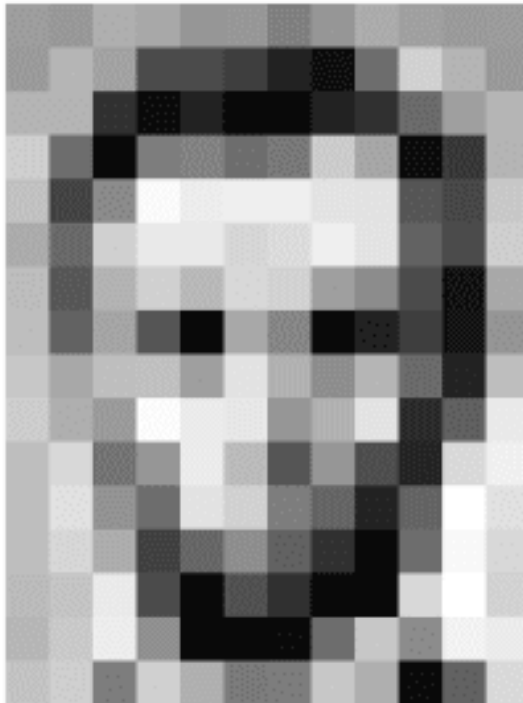
Object Detection

Cat, Dog



Image Segmentation

Image-based Tasks



157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	34	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	215	116	149	236	187	85	150	79	38	218	241
190	234	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	95	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	95	218

157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	34	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	215	116	149	236	187	85	150	79	38	218	241
190	234	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	95	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	95	218

For computers, images are just *grids of numbers*.

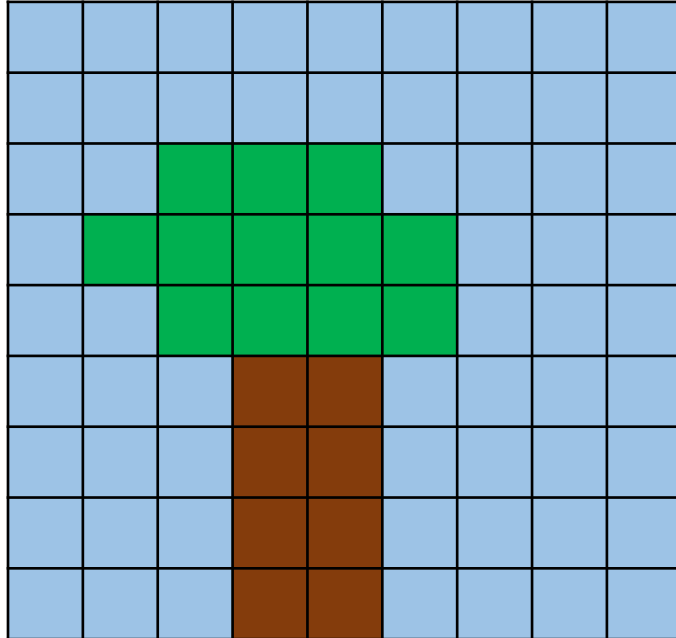
Problems with fully-connected networks



Size:

- 224 x 224 RGB image → **150,528** dimensions
- Hidden layers generally larger than inputs
- One hidden layer = 150,528 x 150,528 weights
= **22 billion!**

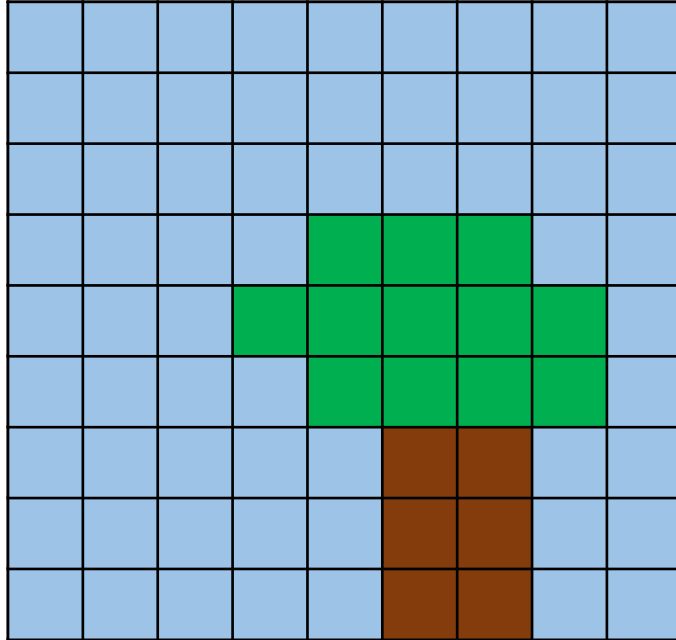
Problems with fully-connected networks



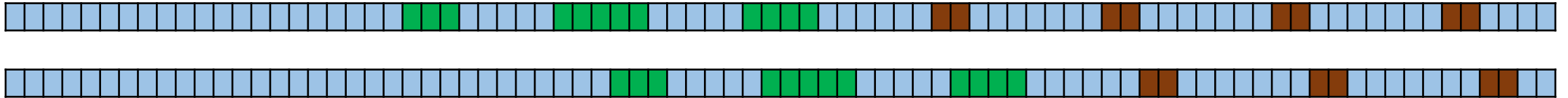
- **Nearby pixels statistically related**
 - FCNNs take inputs as 1D vector
 - no notion of “nearby” in FCNNs and treat relationship between every input equally.



Problems with fully-connected networks



- **Instability under geometric transformations**
 - re-learn same patterns at different parts of image



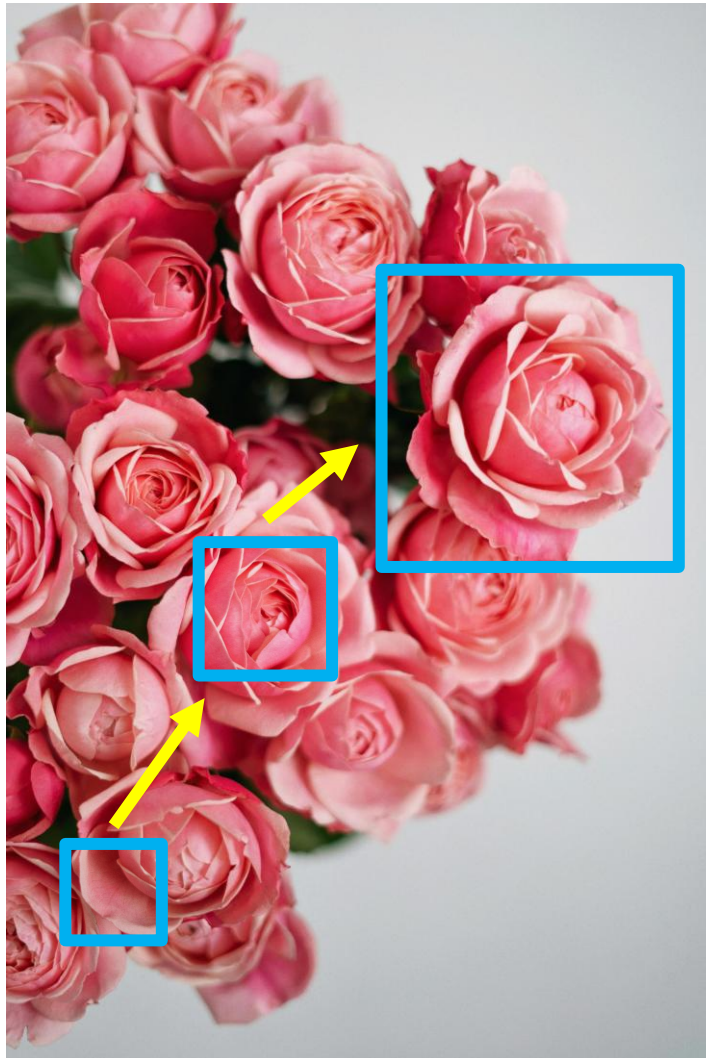
Locality and Translation Invariance



Locality: nearby pixels are more strongly correlated

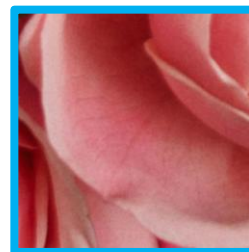
Translation invariance: meaningful patterns can be anywhere in the image

Taking Advantage of Spatial Structure

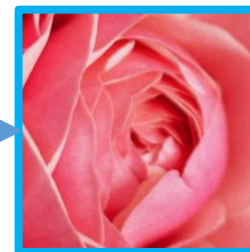


Weight sharing: use the same network parameters to identify local patterns at different locations in the image

Hierarchy: local low-level features are composed into larger, more abstract features



**Edges and
textures**

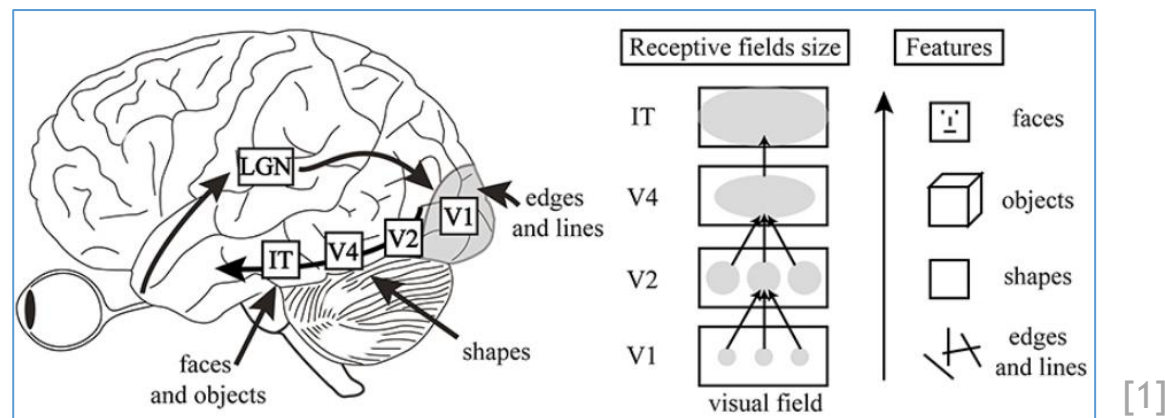
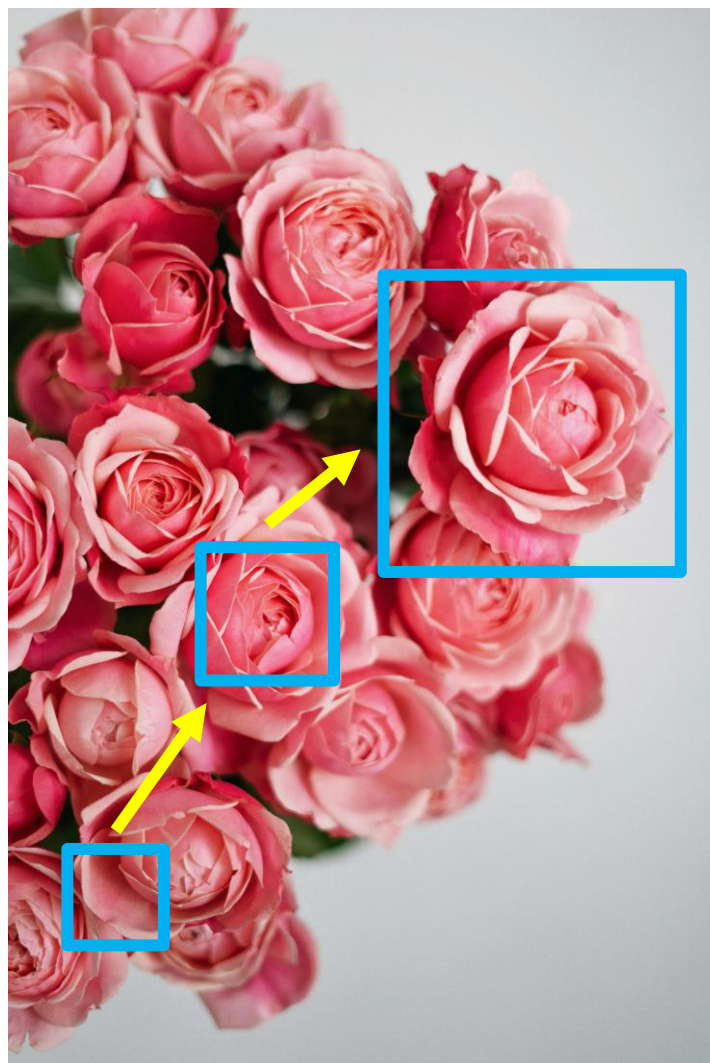


Object parts

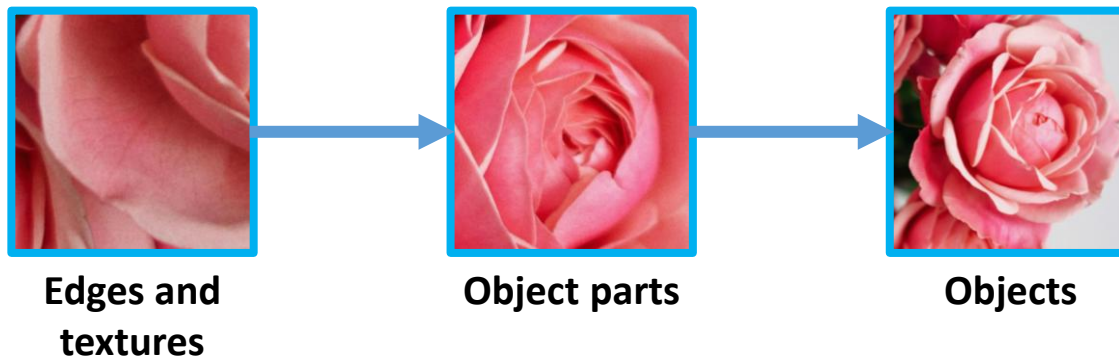


Objects

Taking Advantage of Spatial Structure

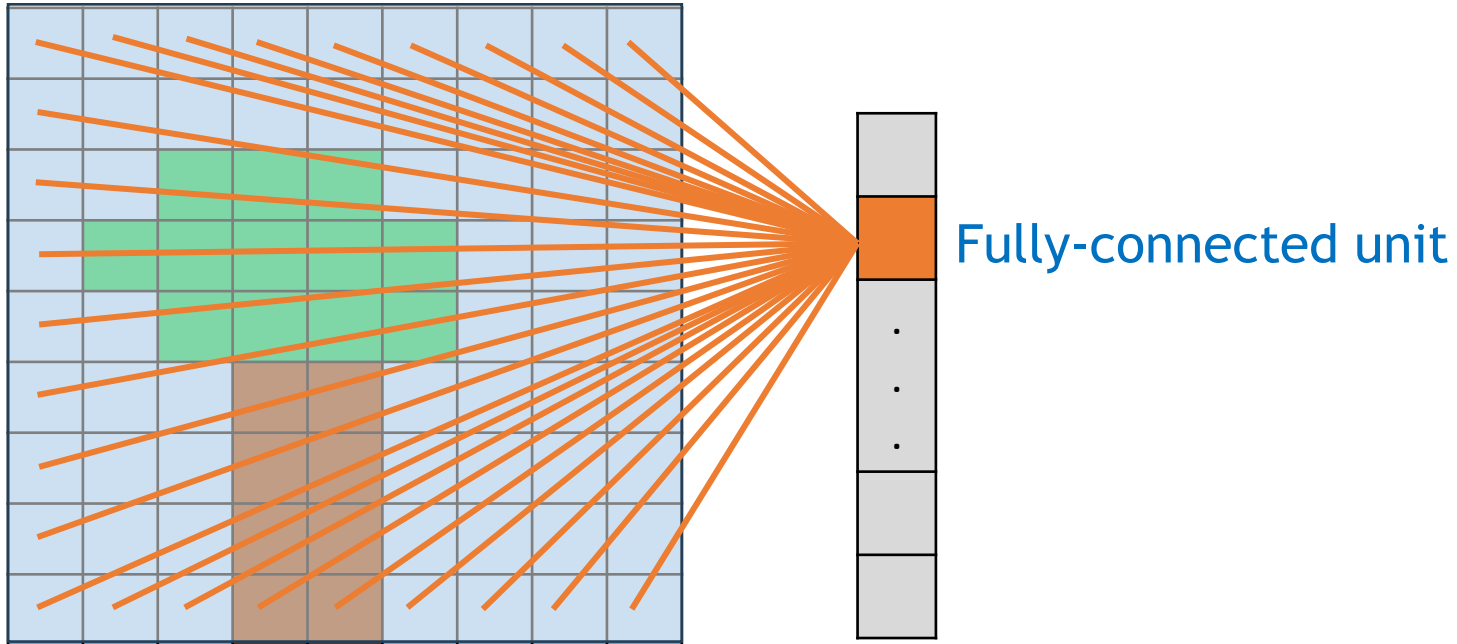


Visual cortex works in a similar manner



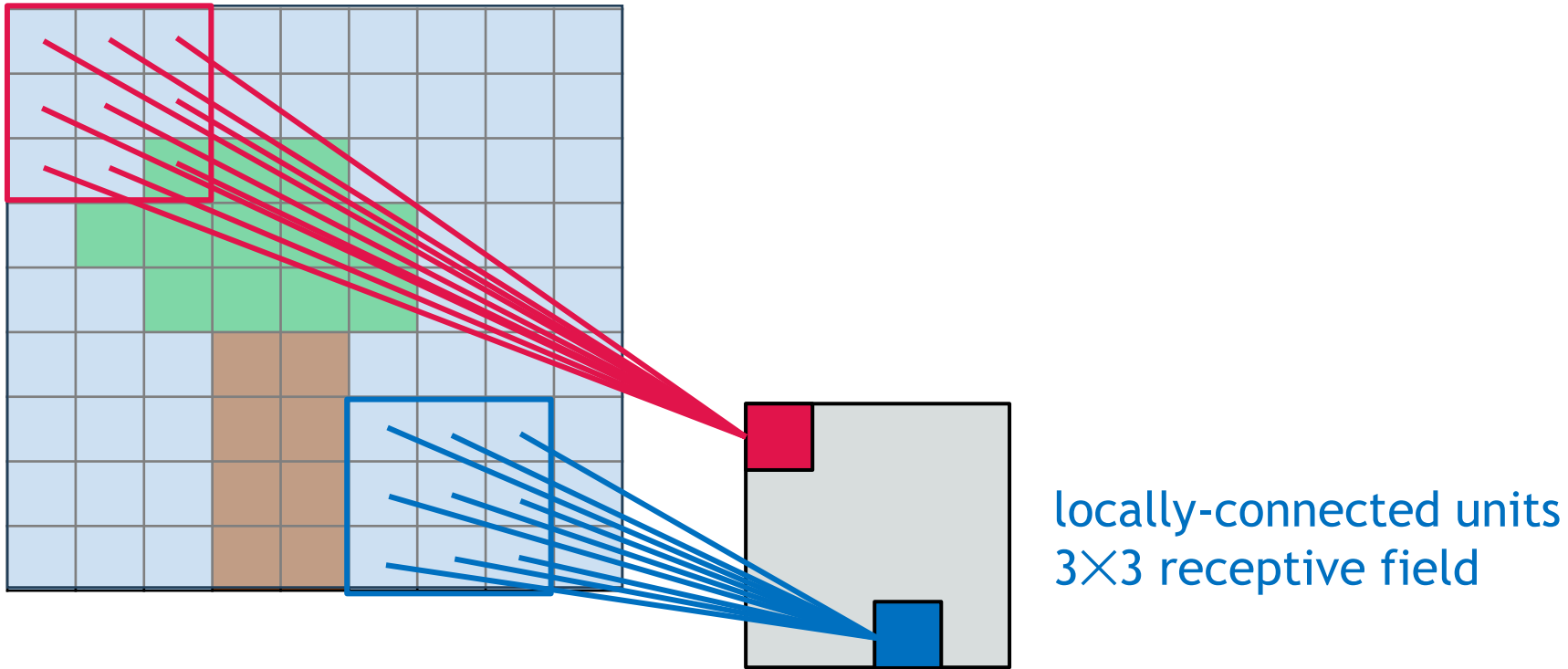
Convolution: Building Blocks

Fully Connected → Locally Connected



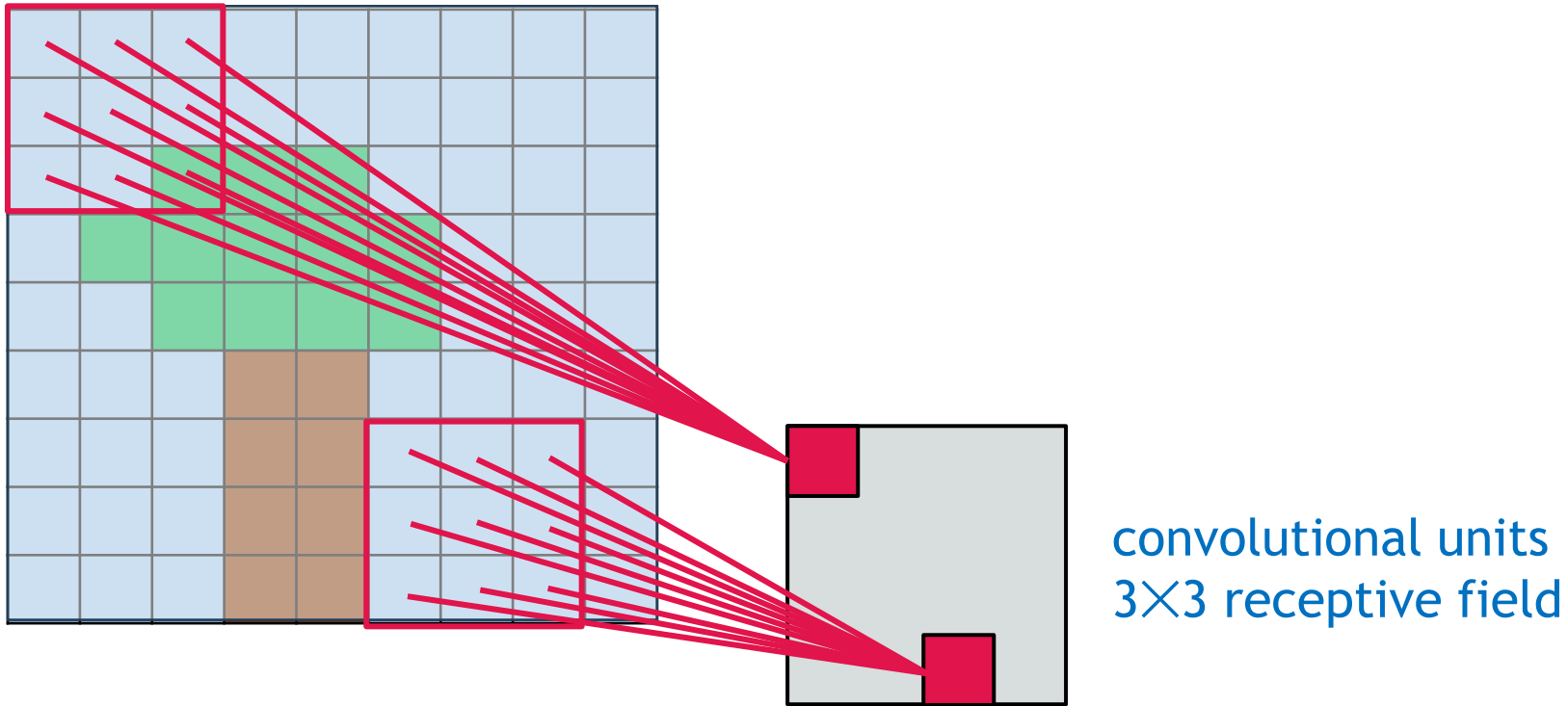
$$y = \sum_{i \in \text{image}} \mathbf{w}_i \mathbf{x}_i + b$$

Fully Connected → Locally Connected



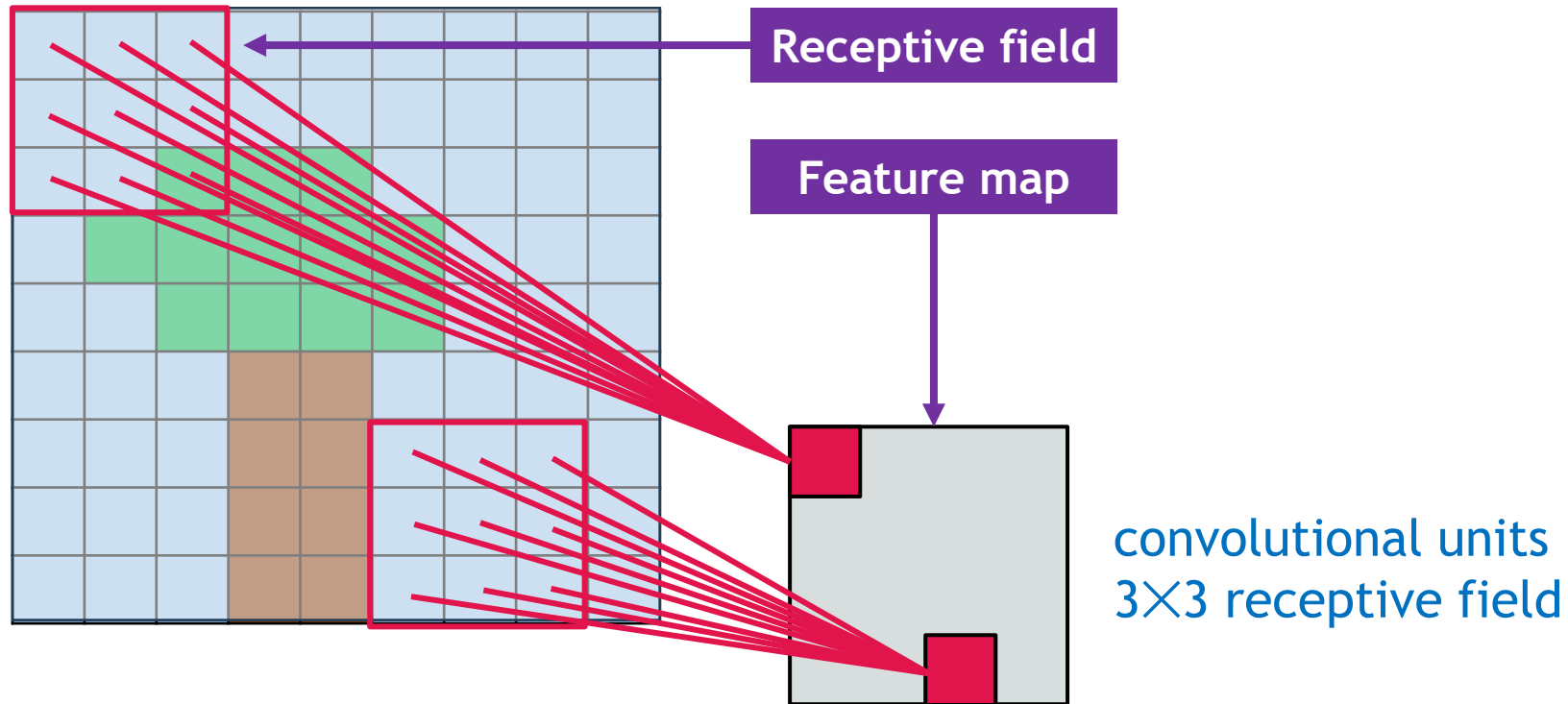
$$y = \sum_{i \in 3 \times 3} \mathbf{w}_i \mathbf{x}_i + b$$

Locally Connected → Convolutional



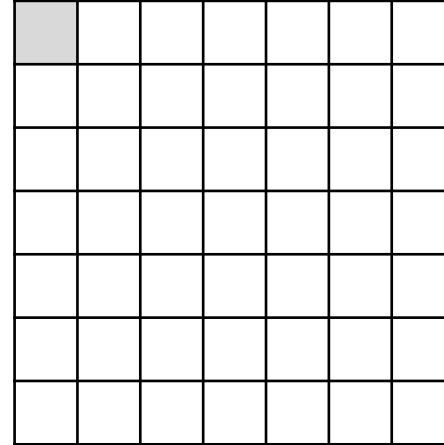
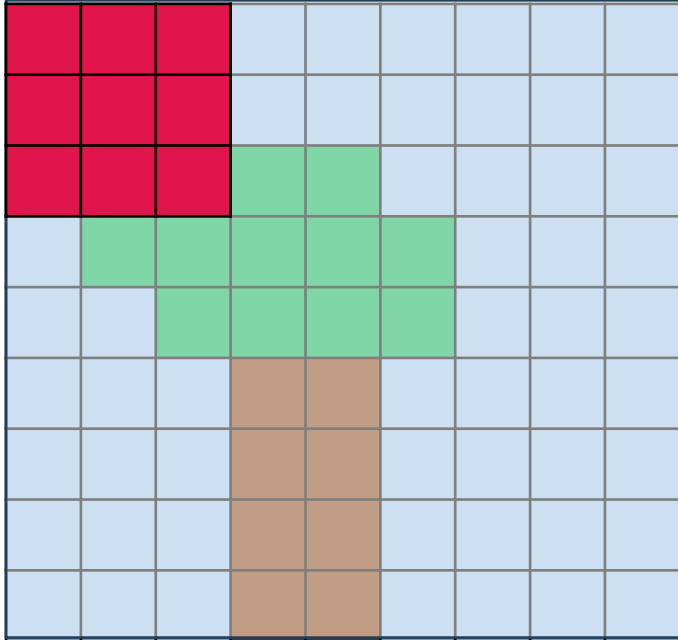
$$y = \mathbf{w} * \mathbf{x} + b$$

Locally Connected → Convolutional



$$y = \mathbf{w} * \mathbf{x} + b$$

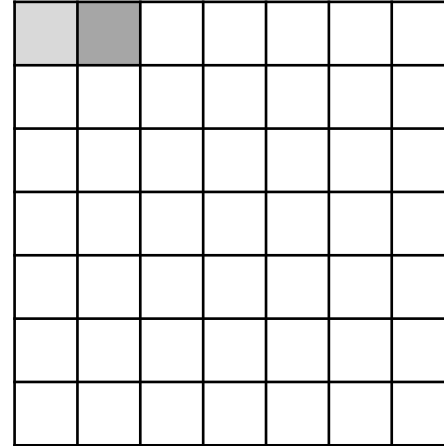
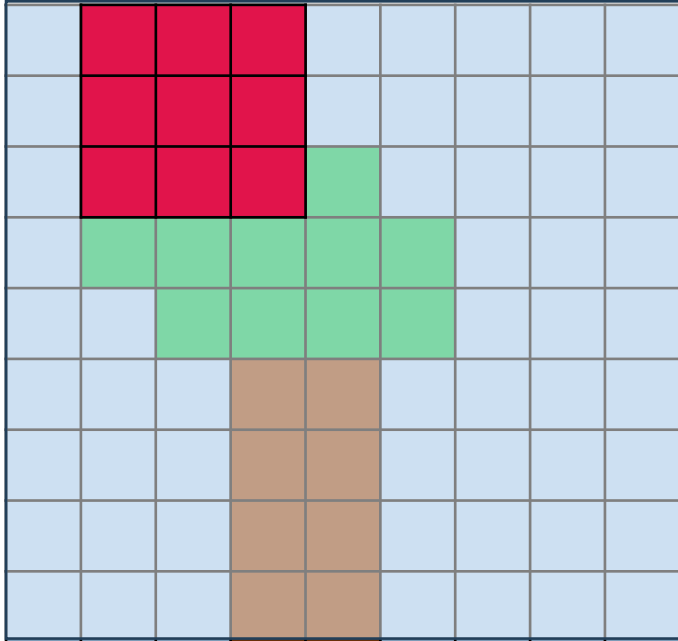
The Convolution Operation



The kernel slides across the image and produces an output value at each position of the feature map

- Slide the kernel over the input image
- Element-wise multiply the values
- Add the products

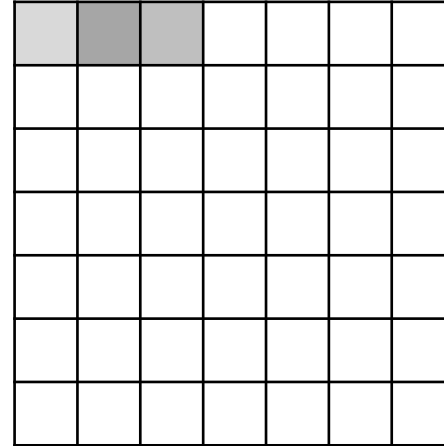
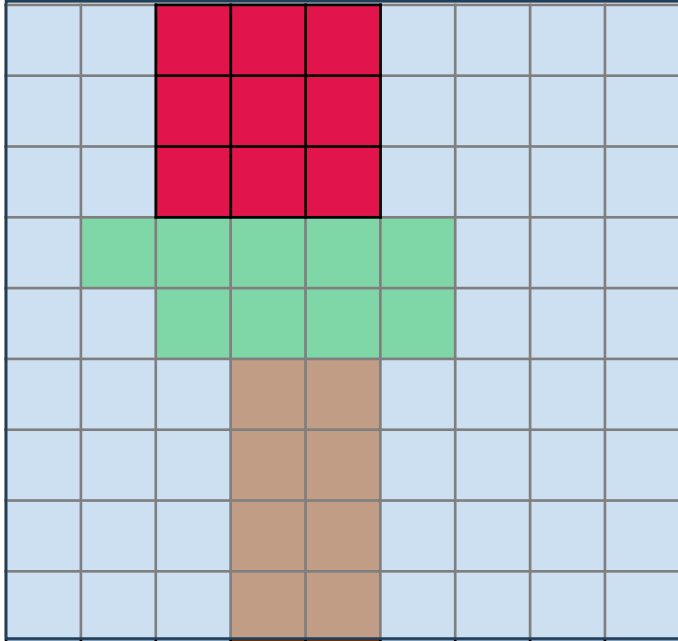
The Convolution Operation



The kernel slides across the image and produces an output value at each position of the feature map

- Slide the kernel over the input image
- Element-wise multiply the values
- Add the products

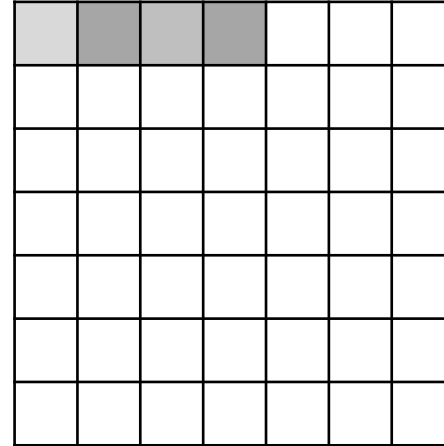
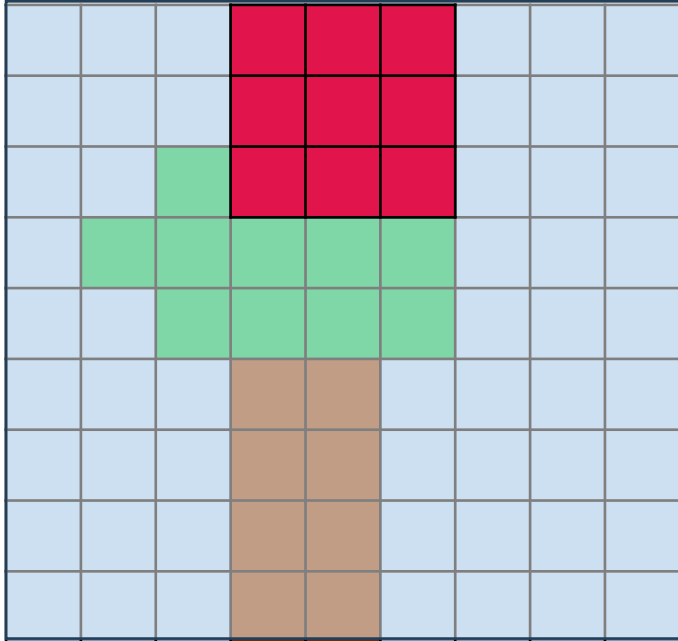
The Convolution Operation



The kernel slides across the image and produces an output value at each position of the feature map

- Slide the kernel over the input image
- Element-wise multiply the values
- Add the products

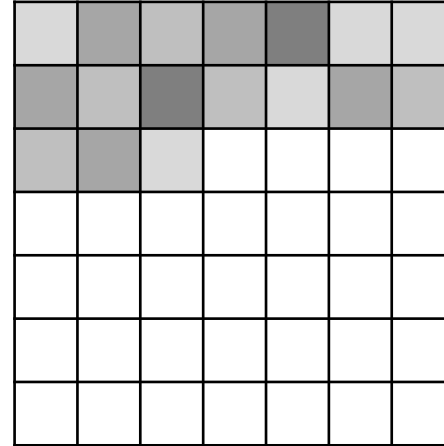
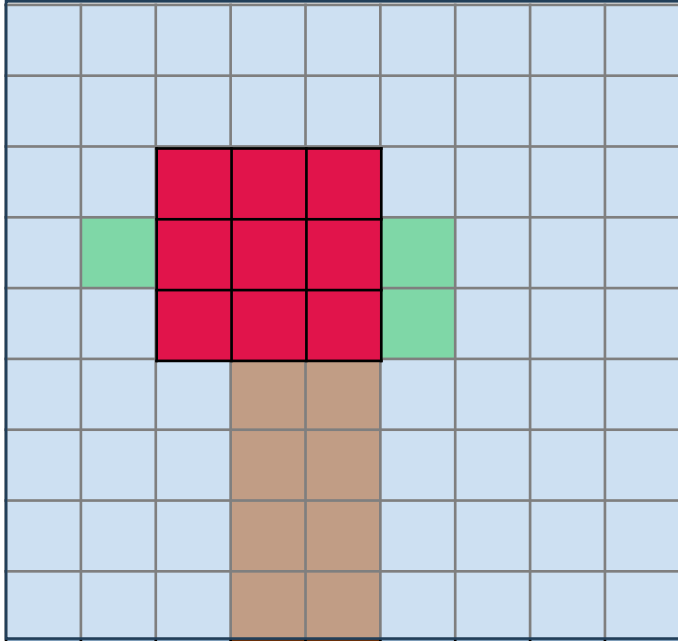
The Convolution Operation



The kernel slides across the image and produces an output value at each position of the feature map

- Slide the kernel over the input image
- Element-wise multiply the values
- Add the products

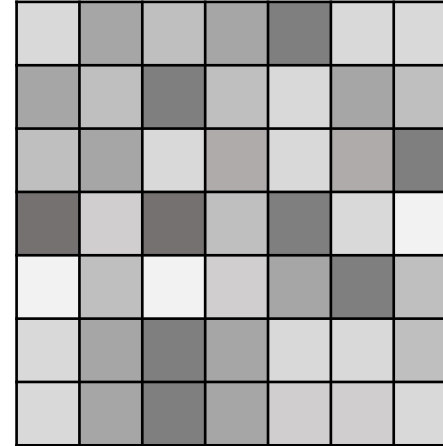
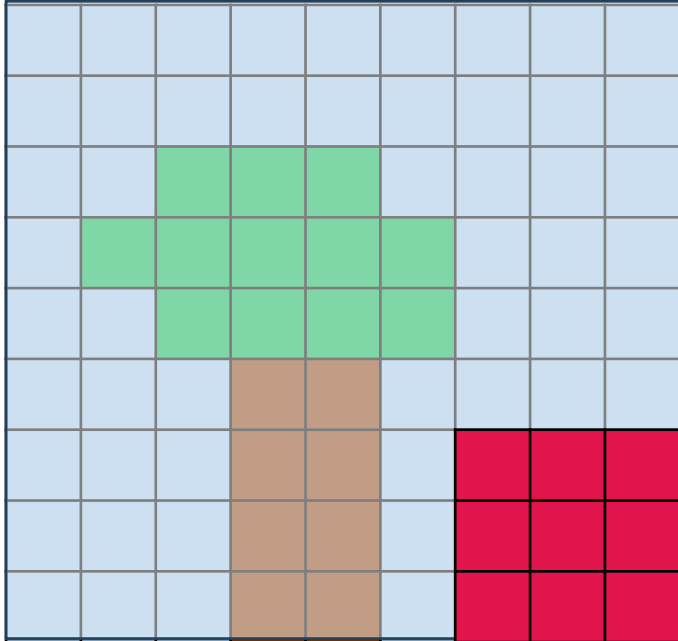
The Convolution Operation



The kernel slides across the image and produces an output value at each position of the feature map

- Slide the kernel over the input image
- Element-wise multiply the values
- Add the products

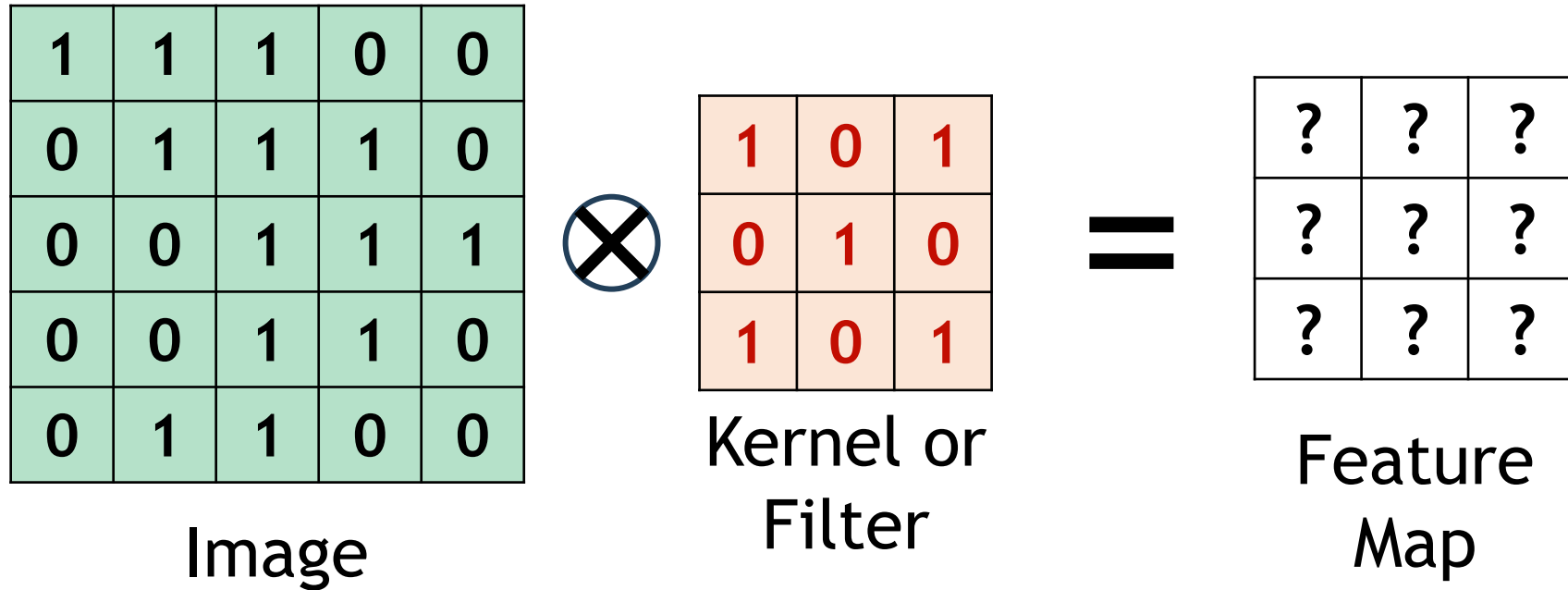
The Convolution Operation



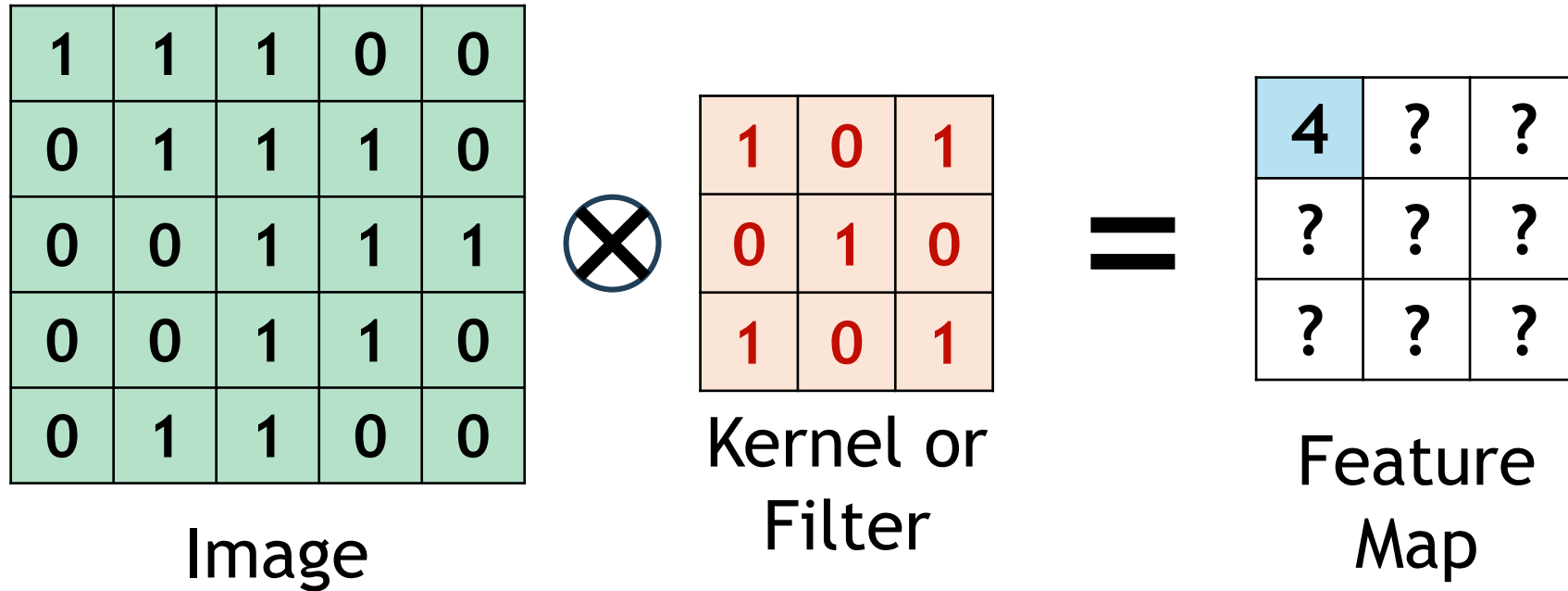
The kernel slides across the image and produces an output value at each position of the feature map

- Slide the kernel over the input image
- Element-wise multiply the values
- Add the products

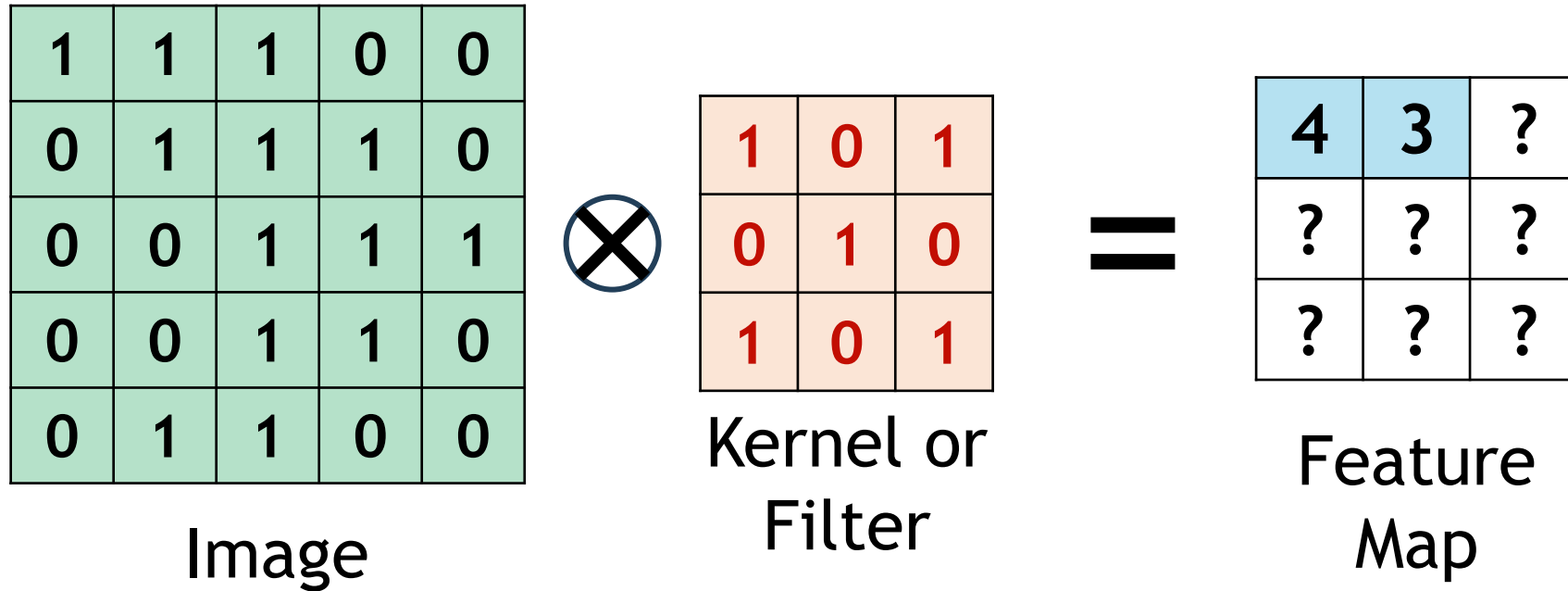
The Convolution Operation



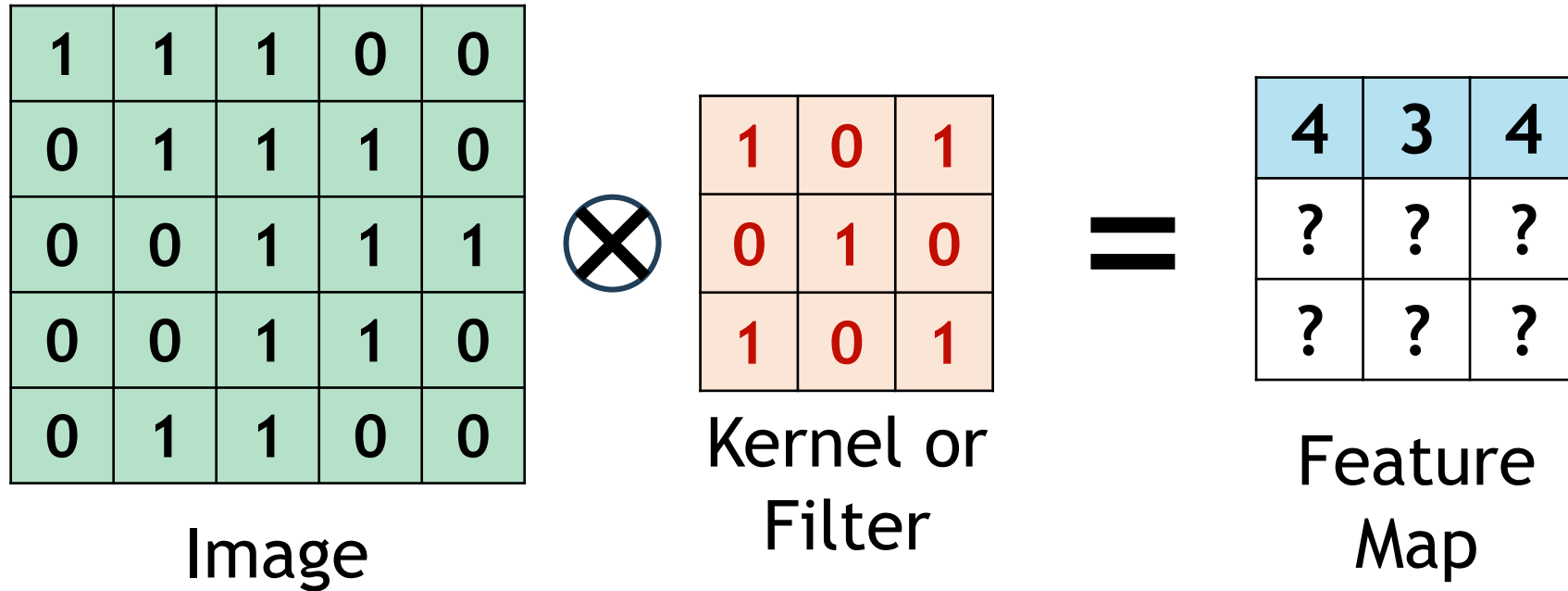
The Convolution Operation



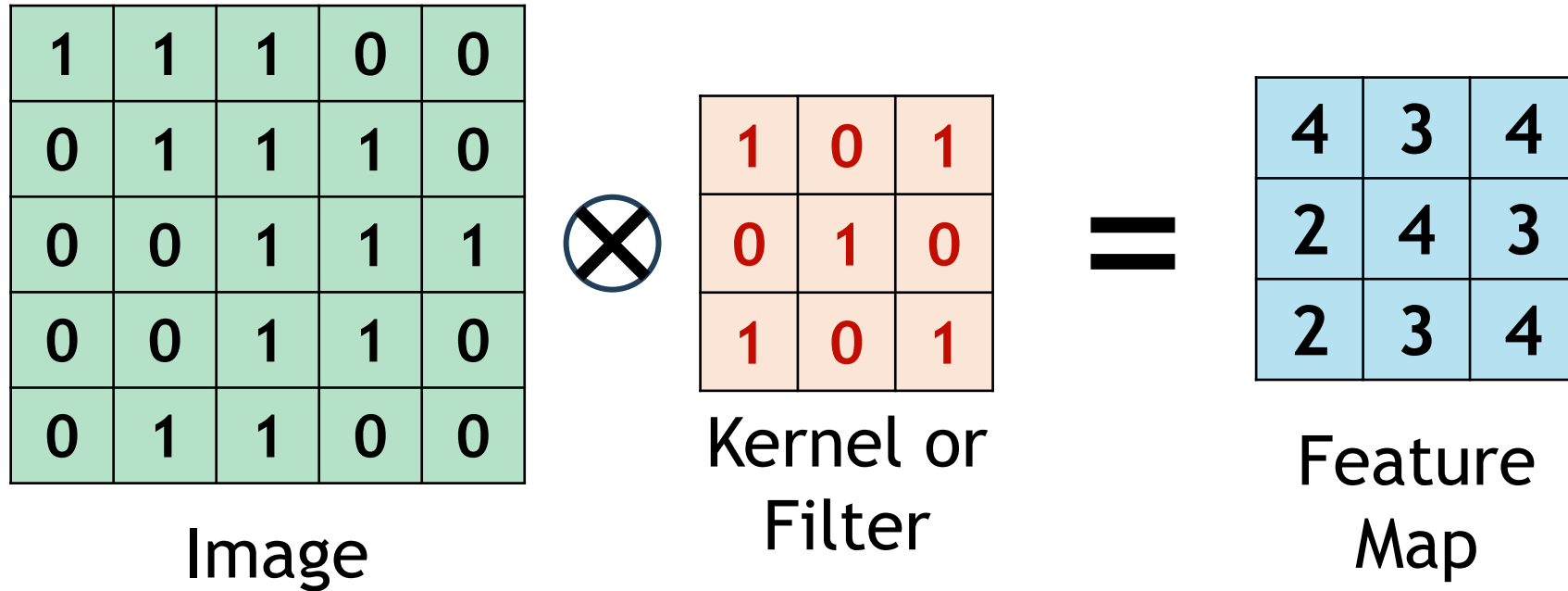
The Convolution Operation



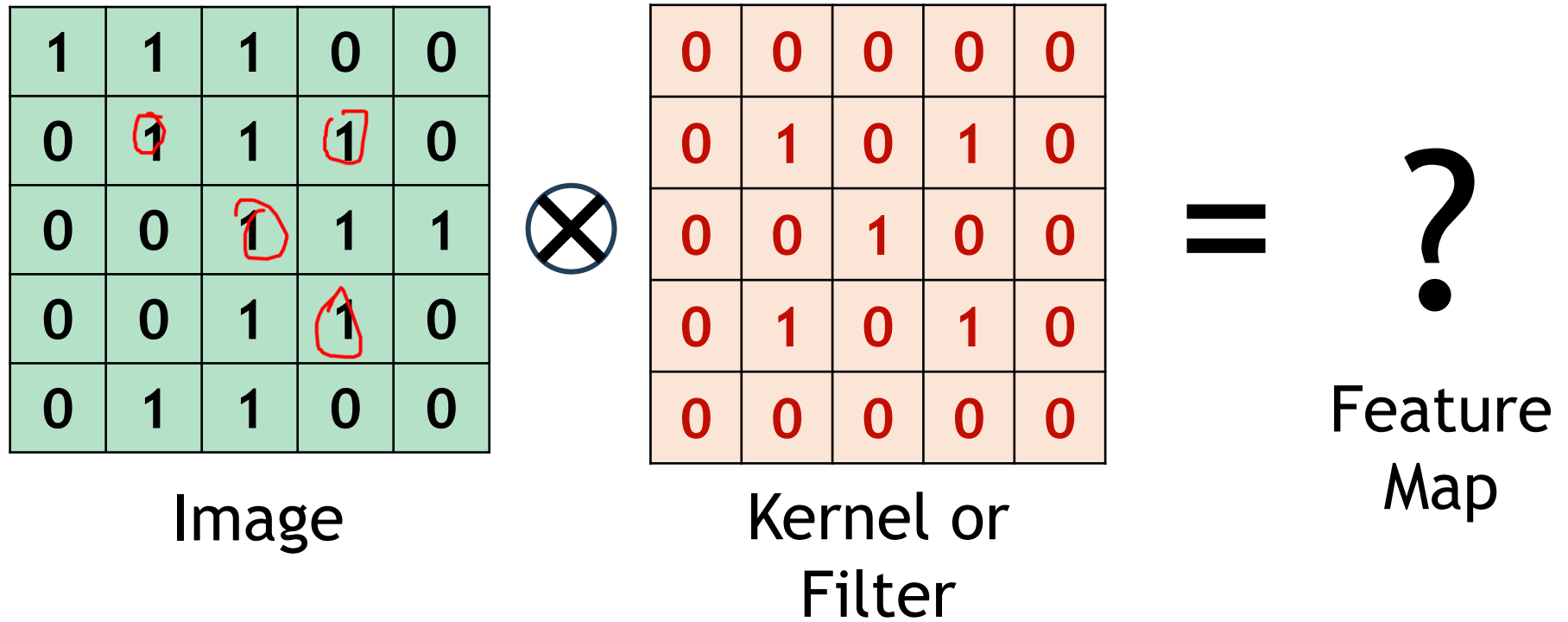
The Convolution Operation



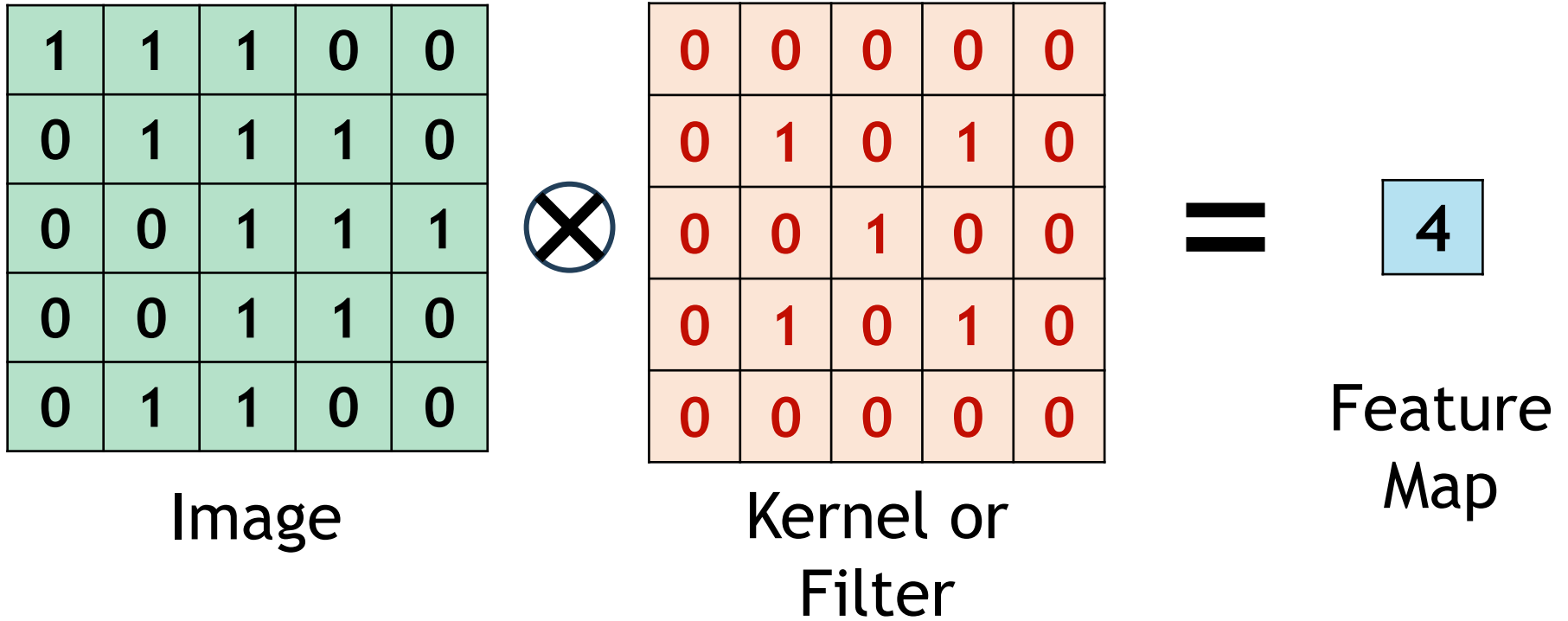
The Convolution Operation



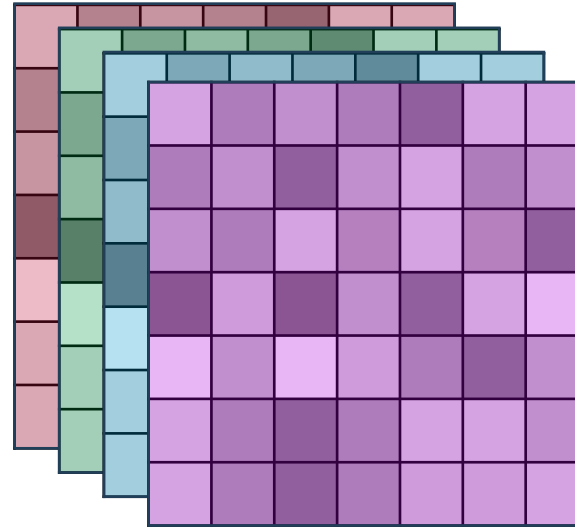
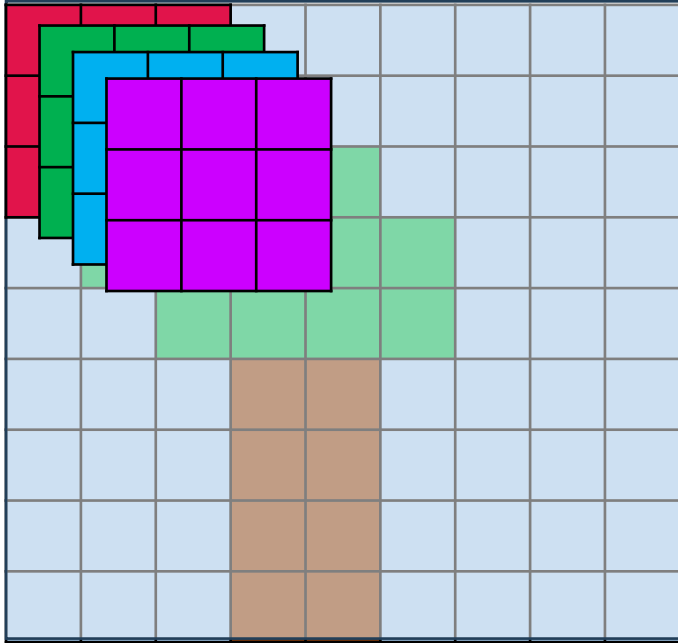
The Convolution Operation



The Convolution Operation



Generating Feature Maps



We convolve multiple kernels and obtain multiple **feature maps** or **channels**

Generating Feature Maps



Original



Sharpen



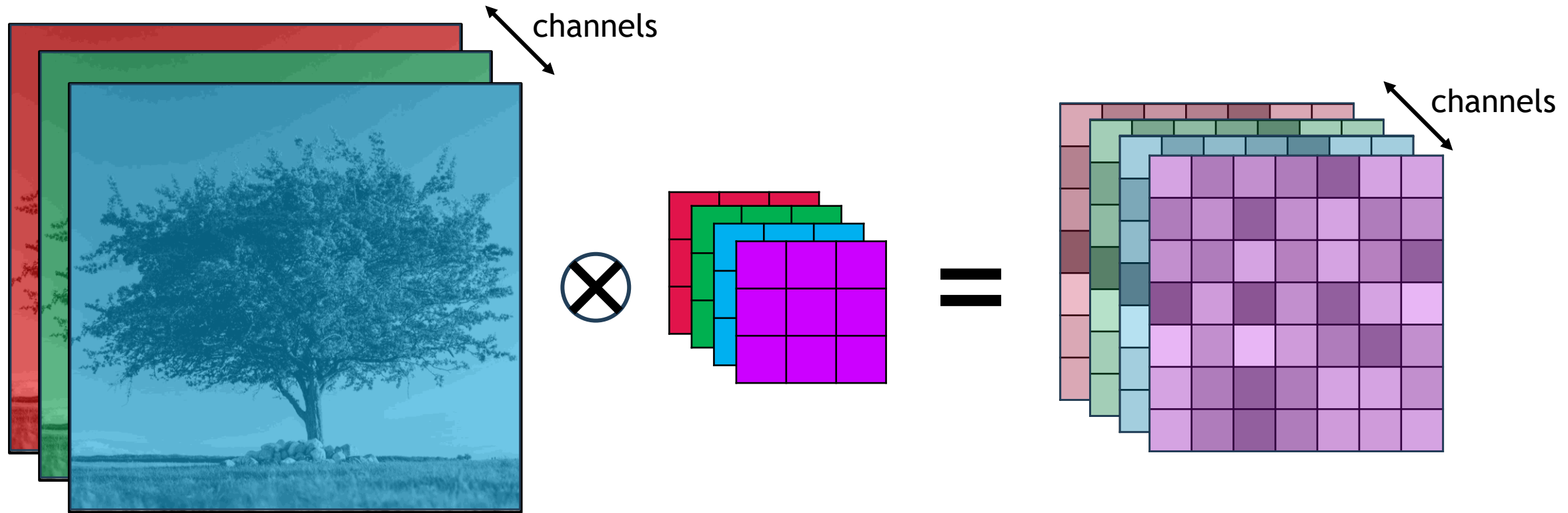
Edge Detect



"Strong" Edge
Detect

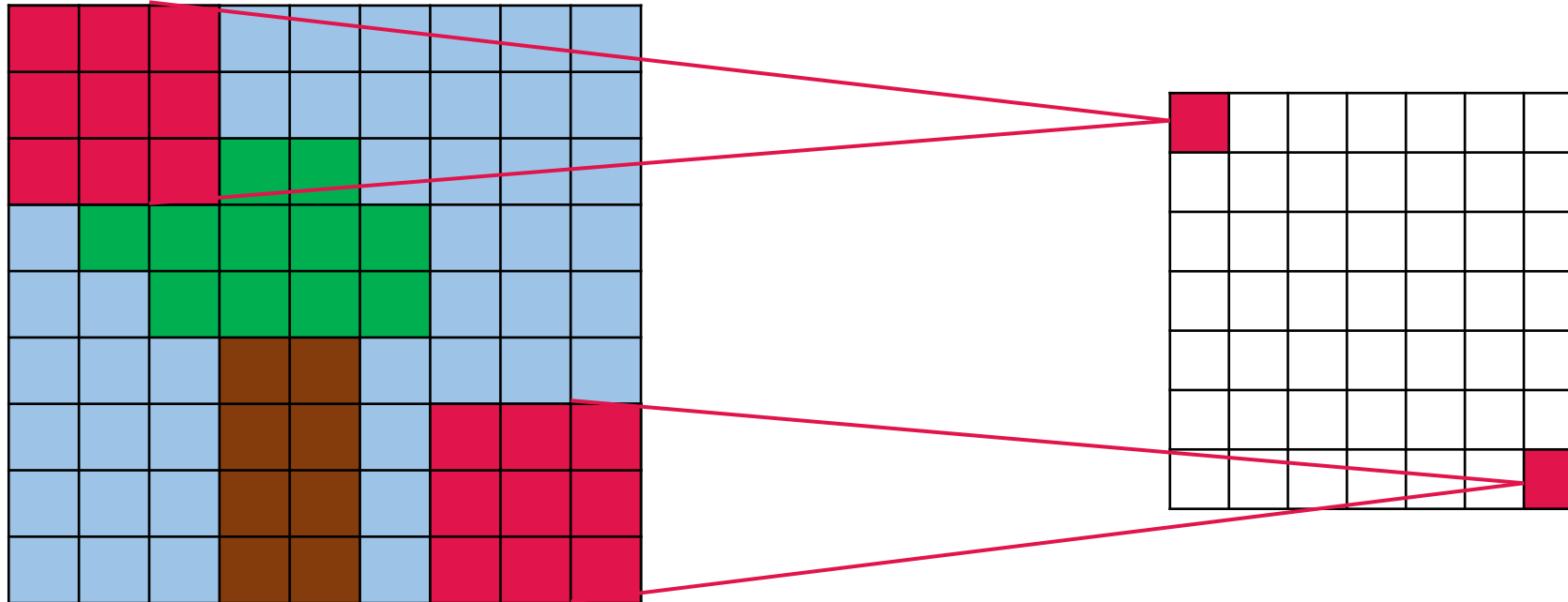
We convolve multiple kernels and obtain multiple **feature maps** or **channels**

Inputs and Outputs are Tensors



Each kernel convolves over all the input channels and generates one feature map

Convolution Variants



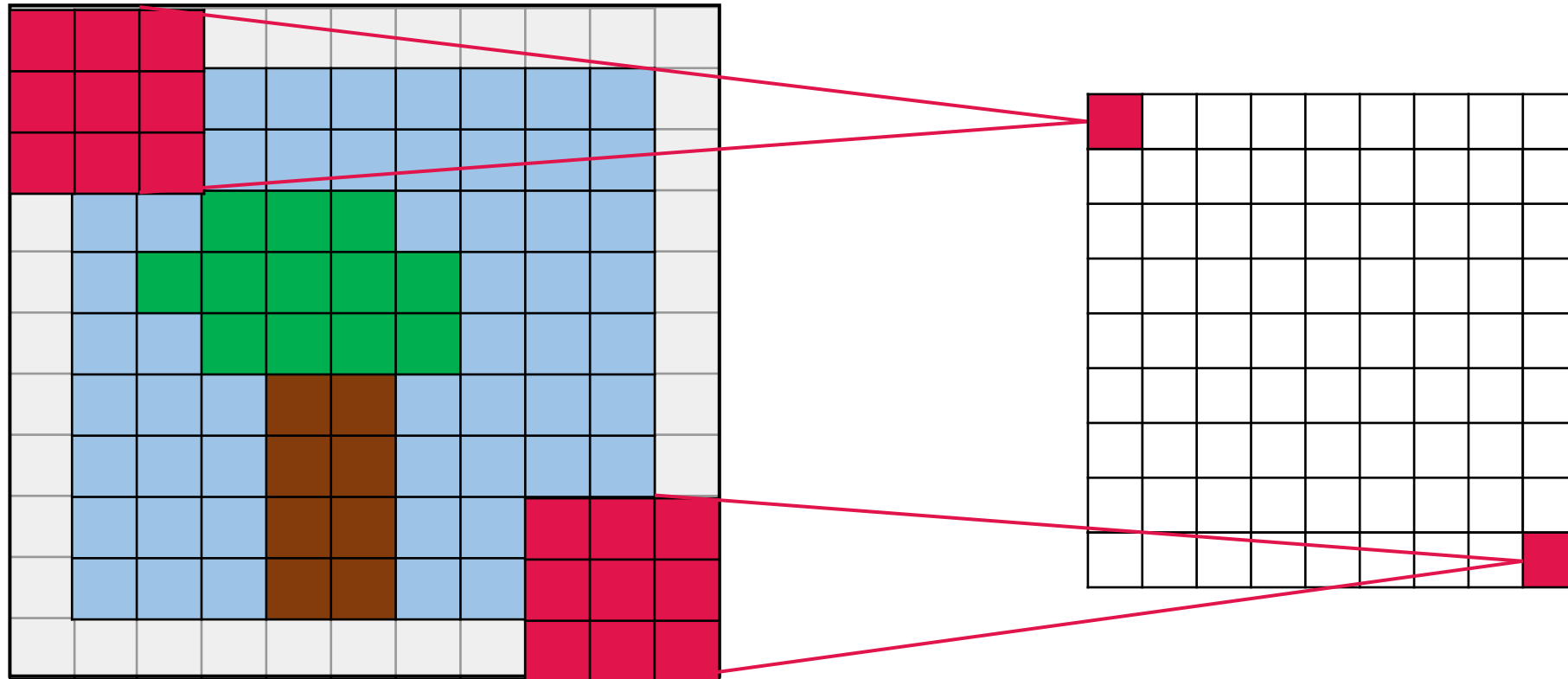
Valid convolution: $\text{output size} = \text{input size} - \text{kernel size} + 1$

Convolution Variants

What if we want the feature map size to match the input image size?

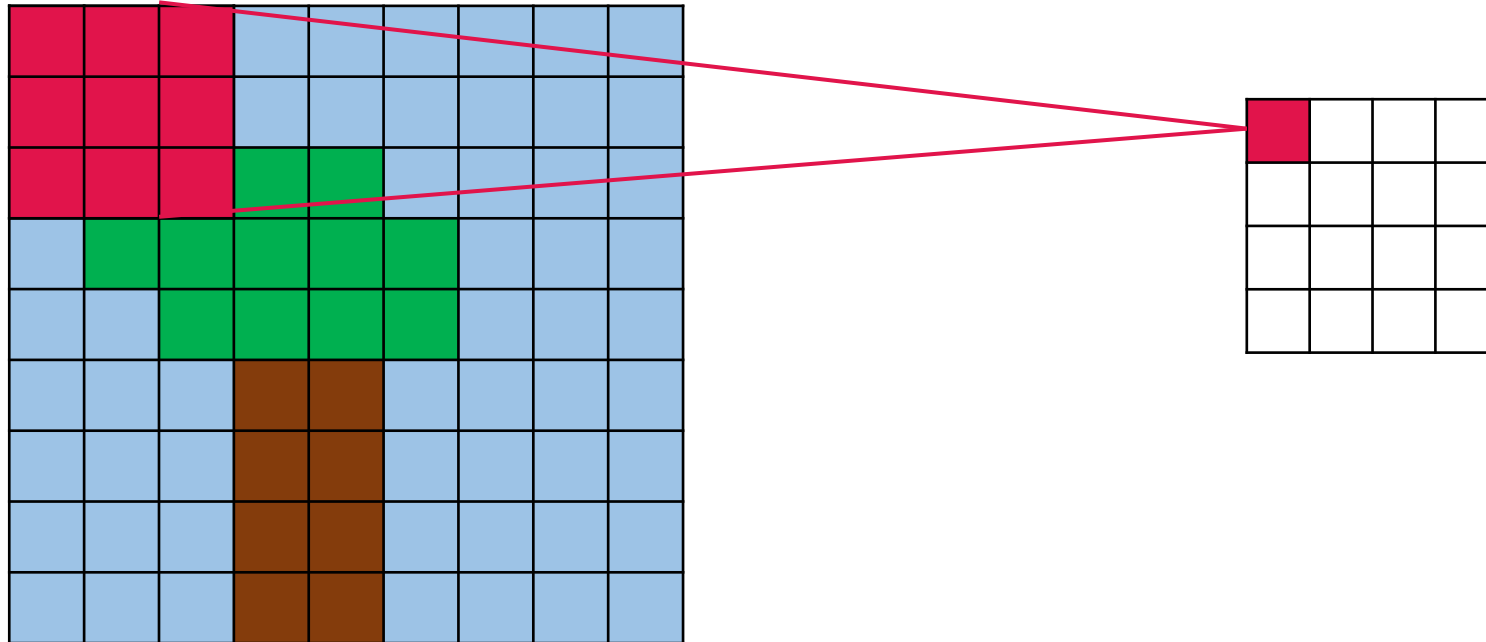
Padding!

Convolution Variants



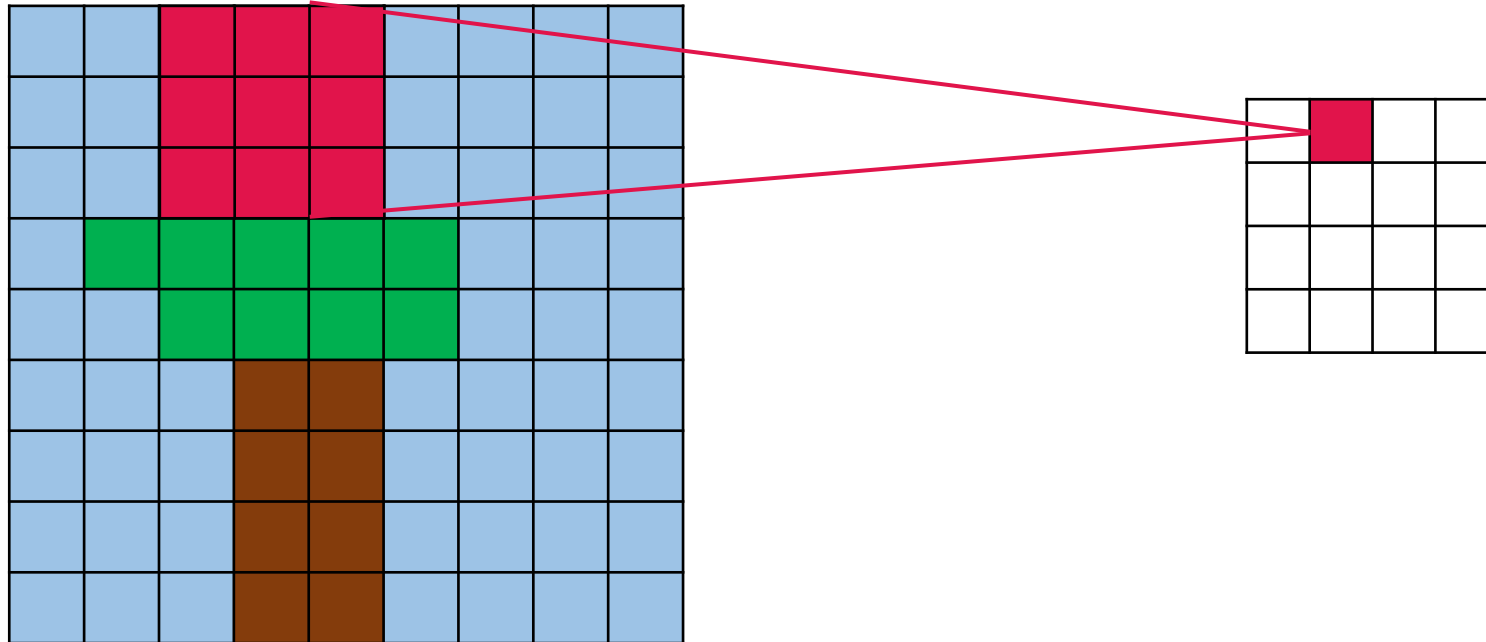
Same convolution: output size = input size

Convolution Variants



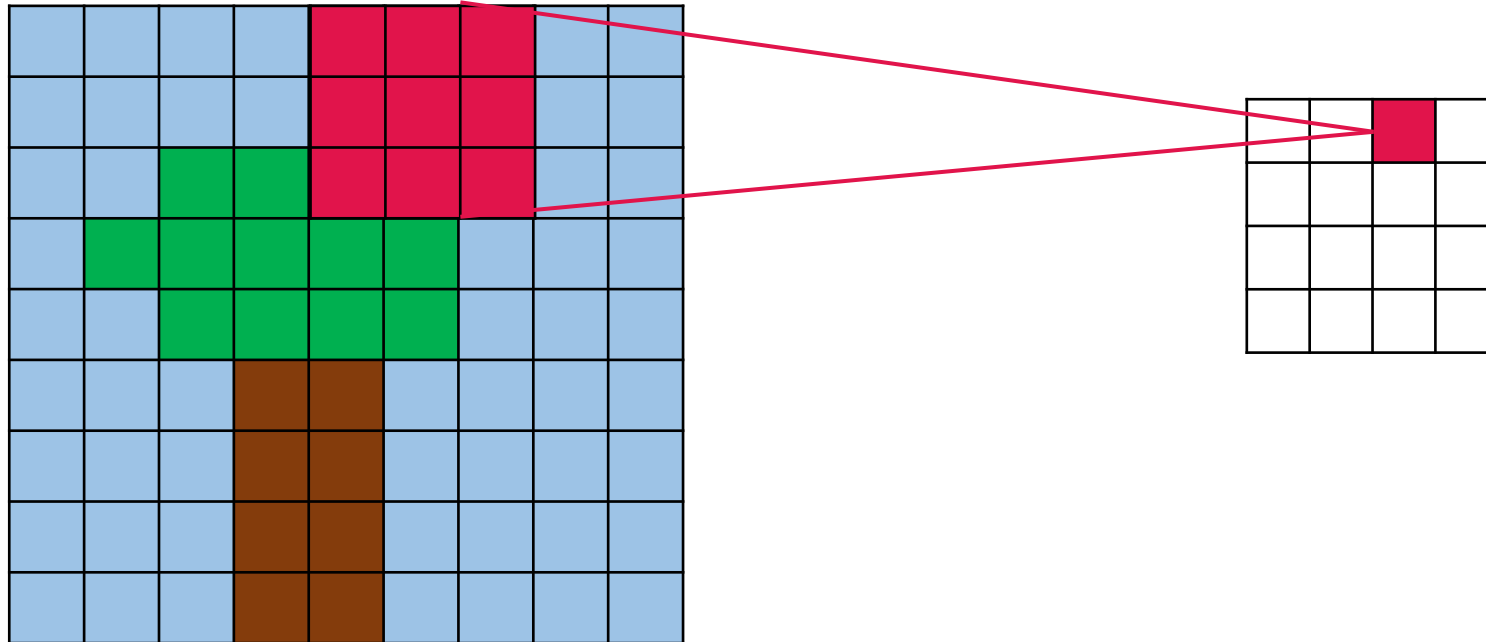
Stride: The number of positions we shift the kernel at each step for each output

Convolution Variants



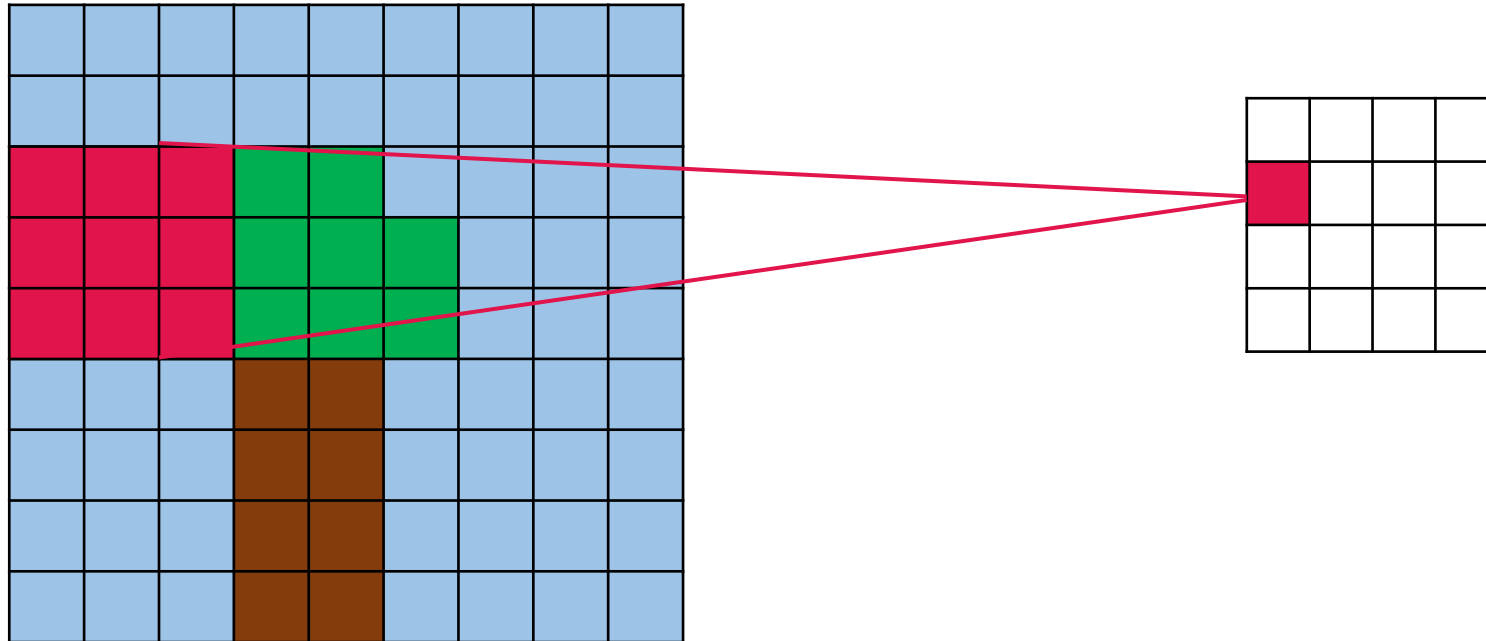
Stride: The number of positions we shift the kernel at each step for each output

Convolution Variants



Stride: The number of positions we shift the kernel at each step for each output

Convolution Variants



Stride: The number of positions we shift the kernel at each step for each output

The Convolution Operation

Valid convolution
Stride = 2

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

Image



1	0	1
0	1	0
1	0	1

Kernel or
Filter

=

?

Feature
Map

The Convolution Operation

Valid convolution
Stride = 2

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

Image



1	0	1
0	1	0
1	0	1

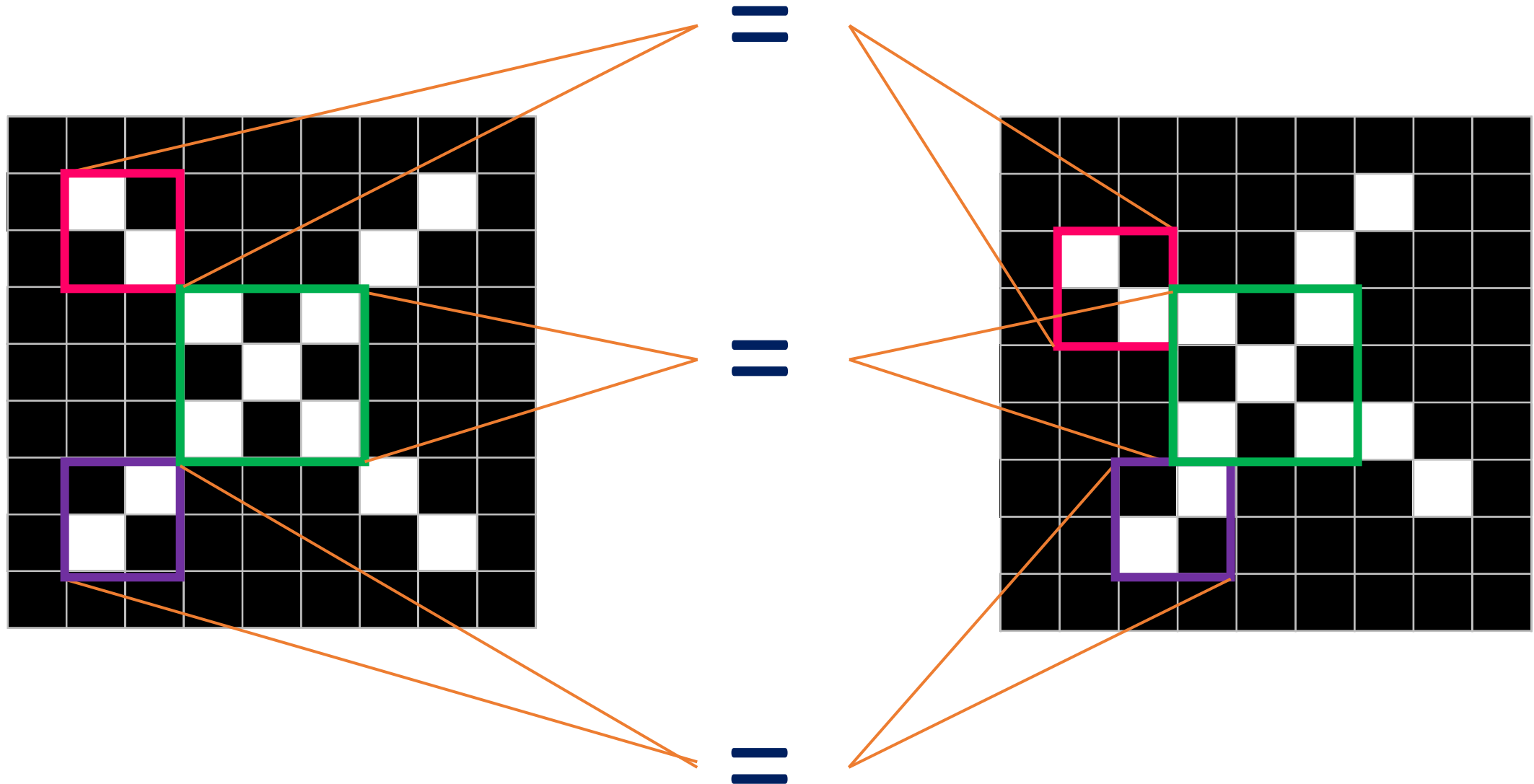
Kernel or
Filter

=

4	4
2	4

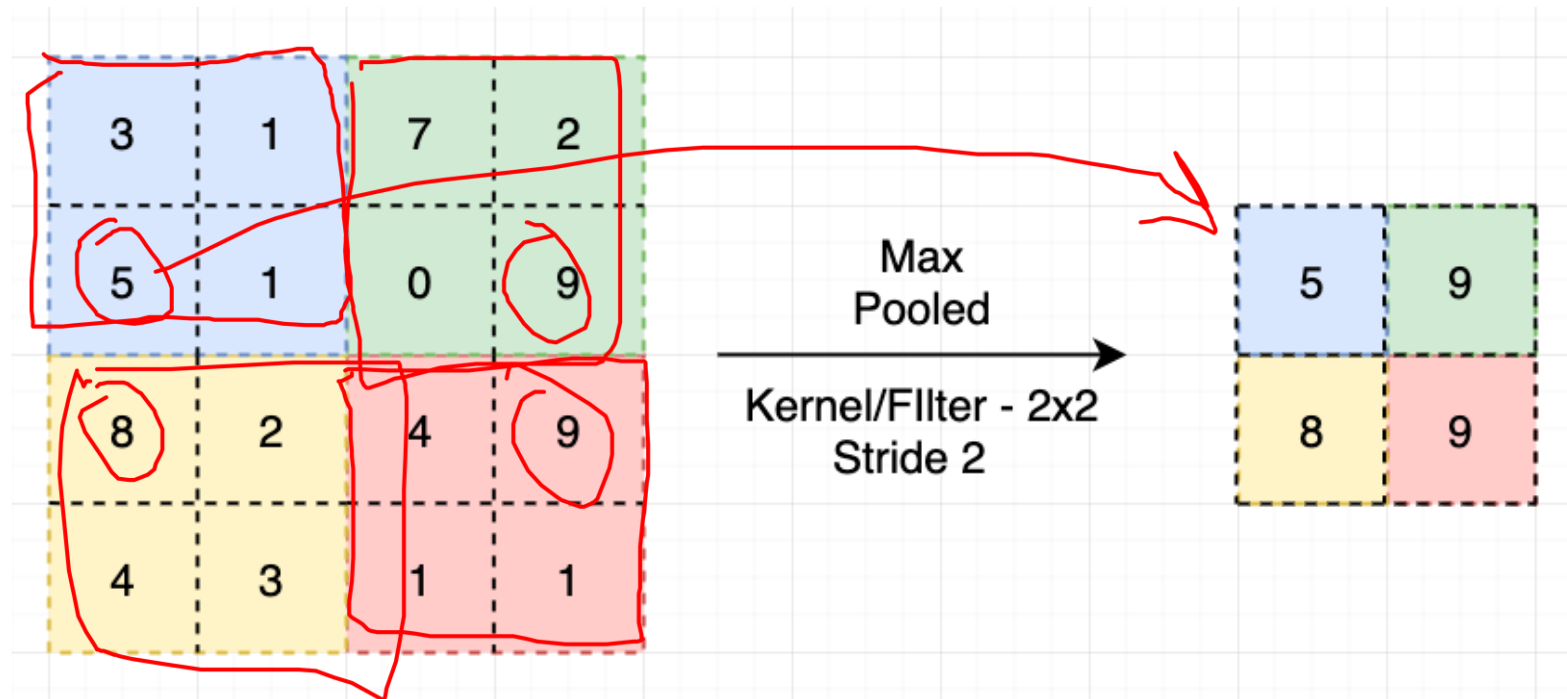
Feature
Map

Convolutions match pieces of the image



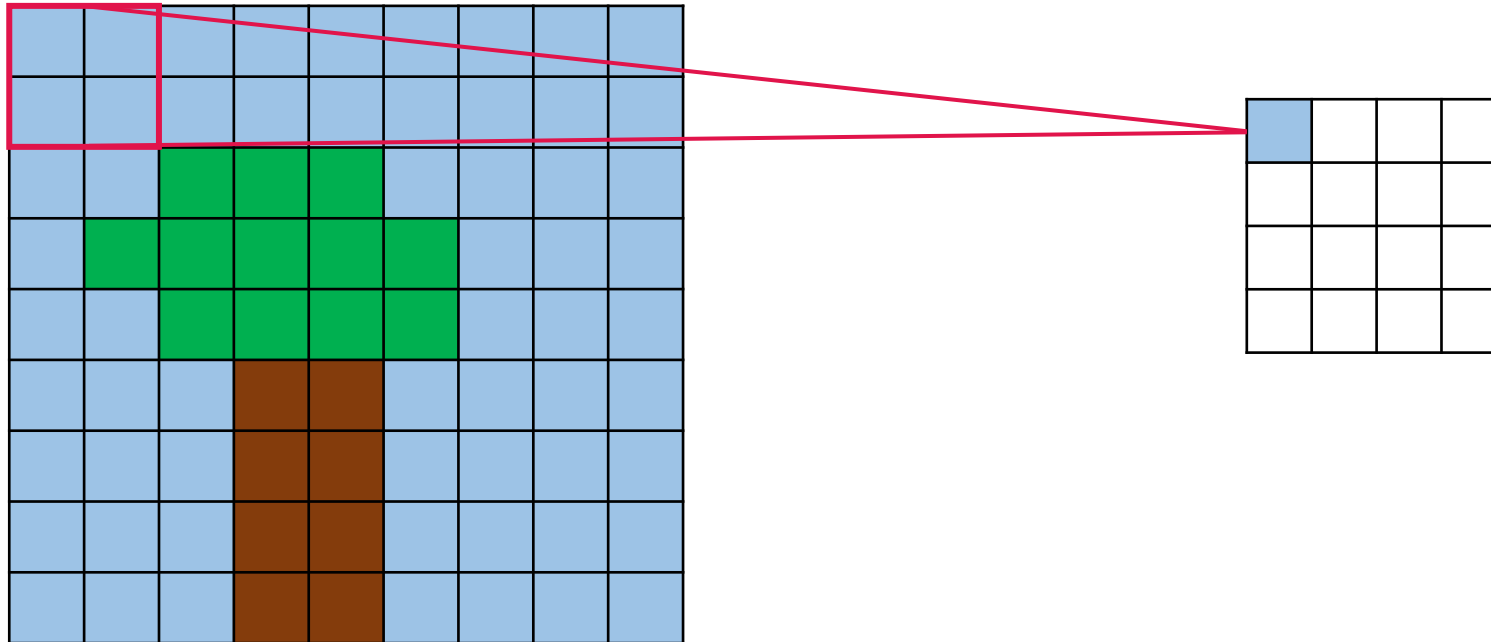
Convolution Variants

Pooling: compute mean or max over small windows to reduce resolution



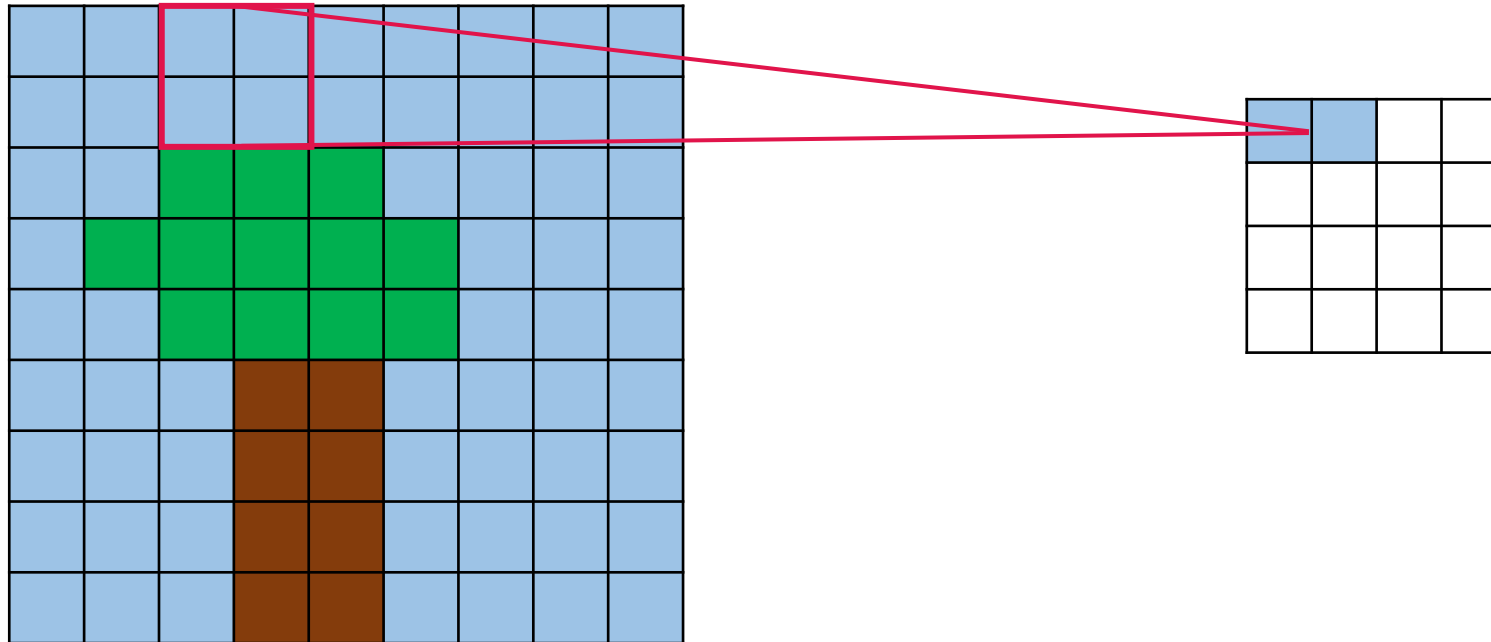
Pooling helps to make the representation **invariant to small translations** of the input

Convolution Variants



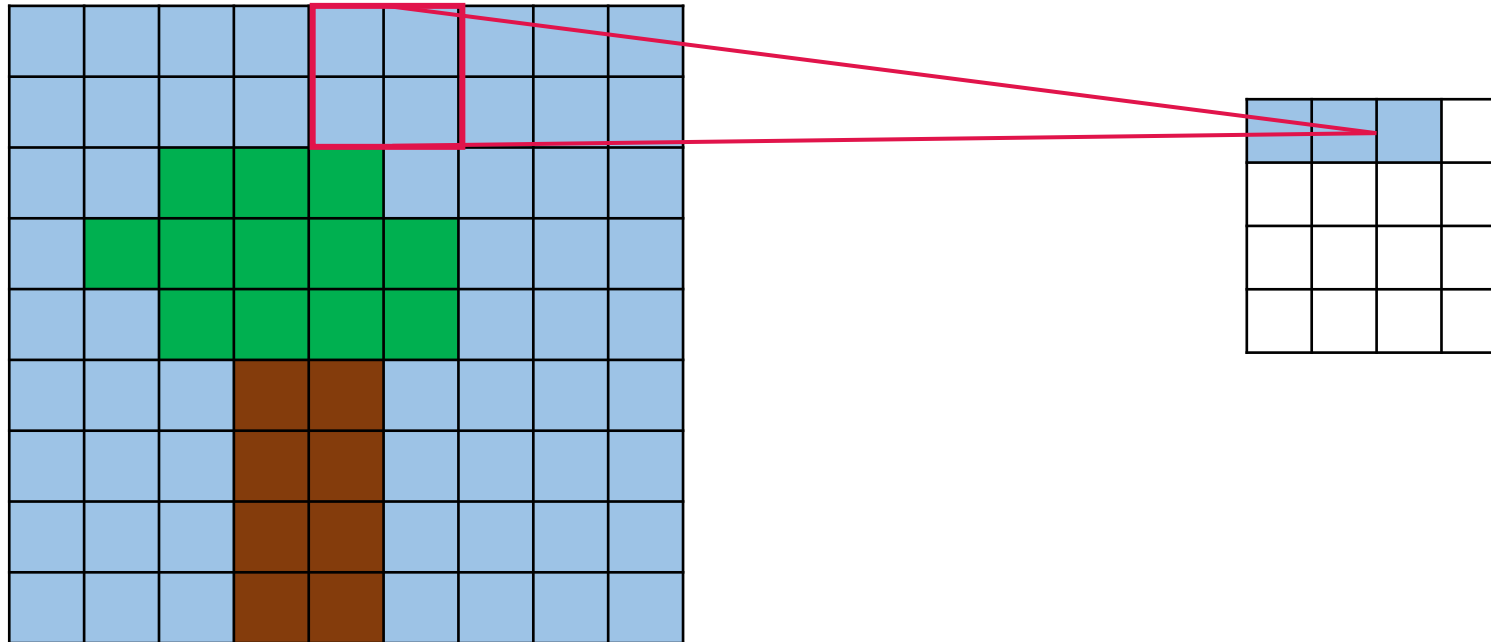
Pooling: compute mean or max over small windows to reduce resolution

Convolution Variants



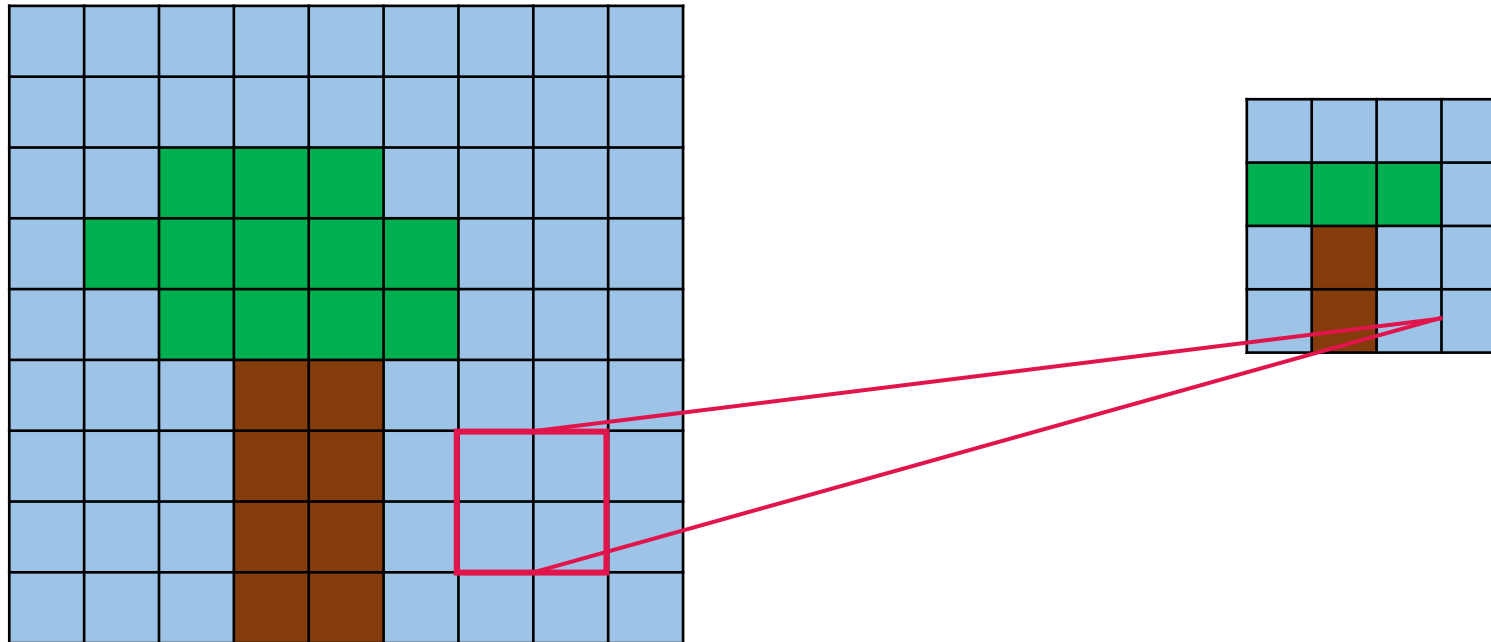
Pooling: compute mean or max over small windows to reduce resolution

Convolution Variants



Pooling: compute mean or max over small windows to reduce resolution

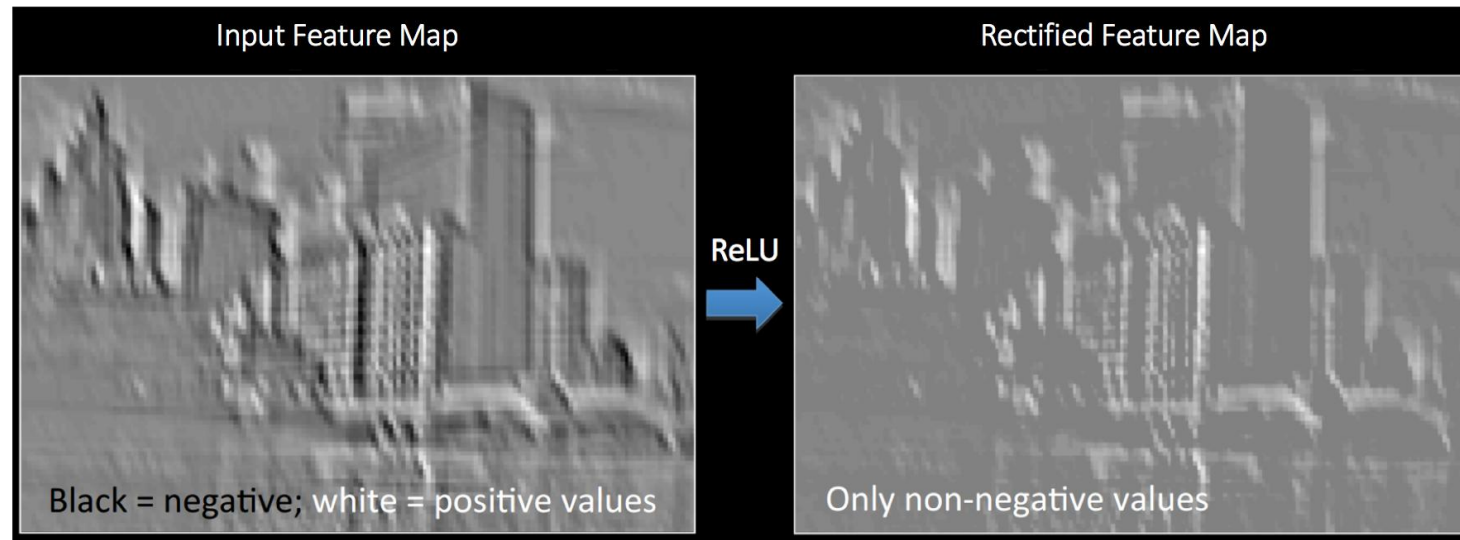
Convolution Variants



Pooling: compute mean or max over small windows to reduce resolution

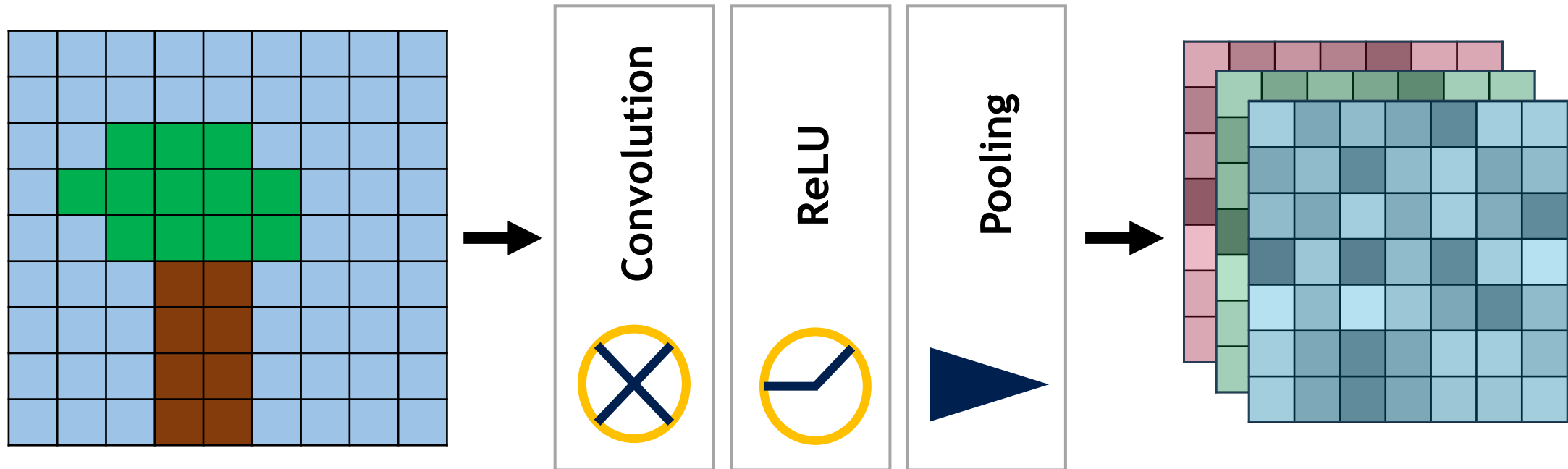
Non-Linearity

- Applied after each convolution operation
- **ReLU:** $f(x) = \max(0, x)$
 - sets all negative values in the feature map to zero. Non-linear operation
- Enables the network to model the non-linear nature of images



Stacking layers

- The output of one layer becomes the input of the next.



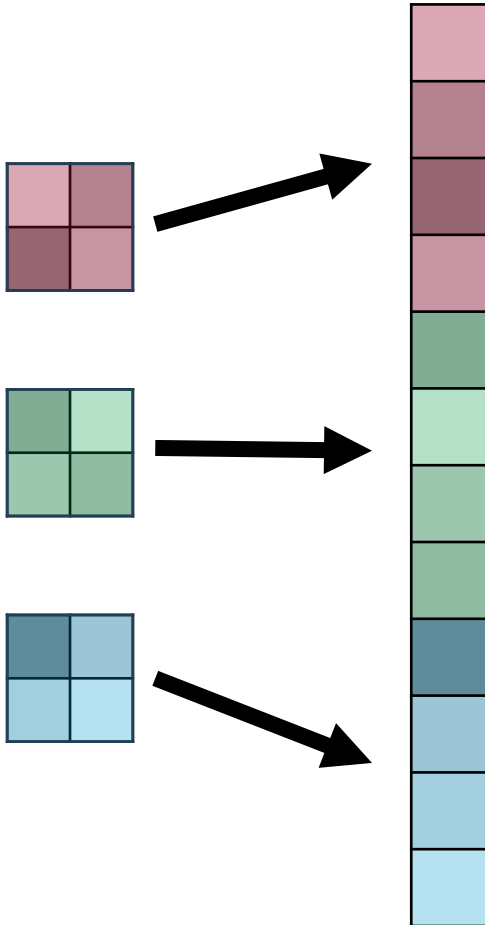
Deep stacks

- Layers can be repeated several times.



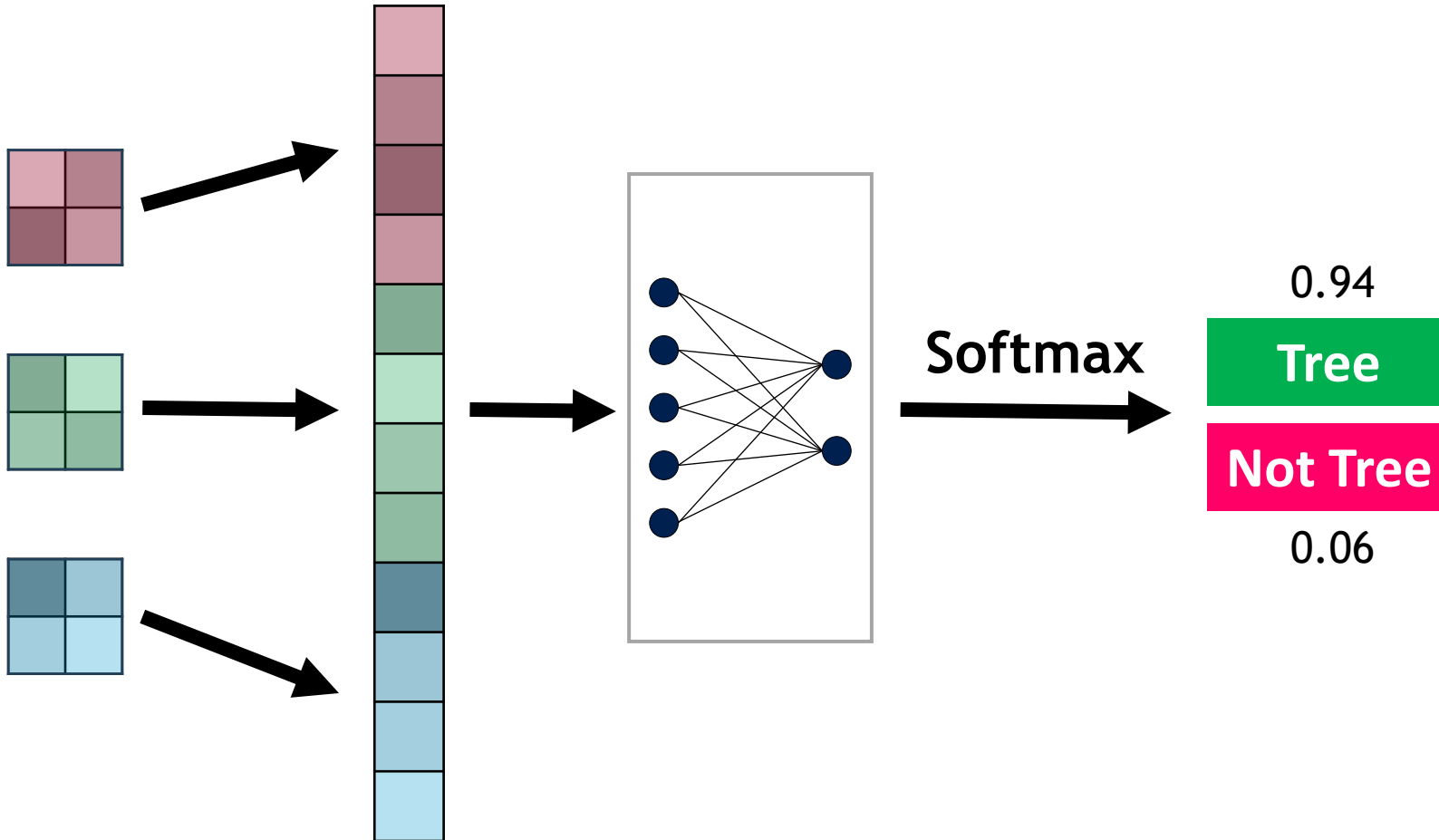
Fully connected layer

- The feature maps are flattened into a vector.



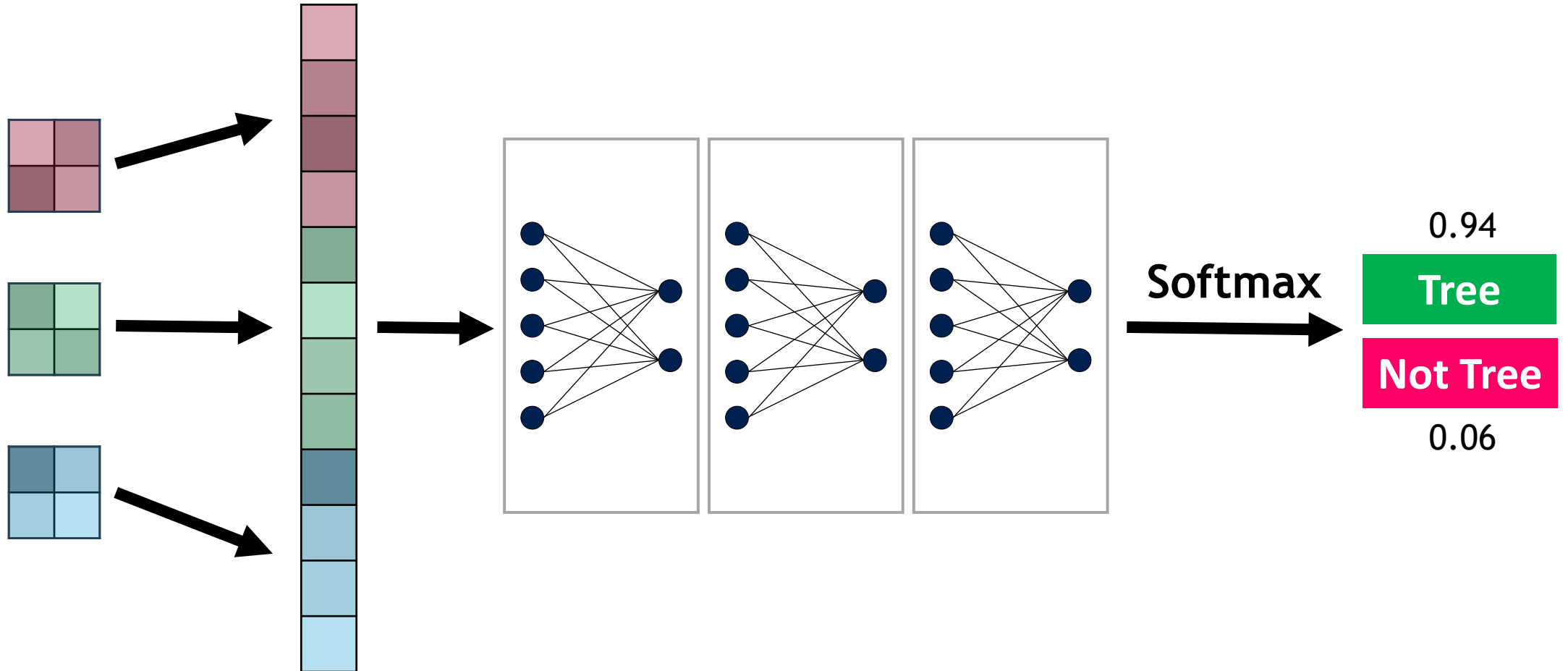
Fully connected layer

- The list of feature values becomes a list of votes.



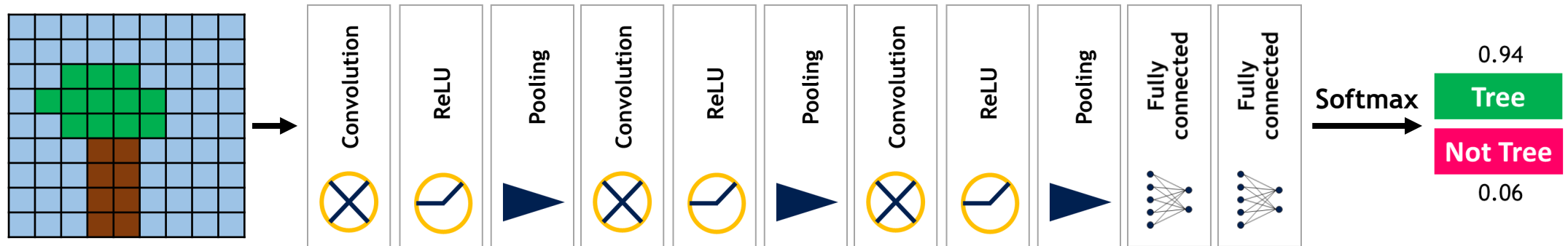
Fully connected layer

- These layers can be stacked as well.



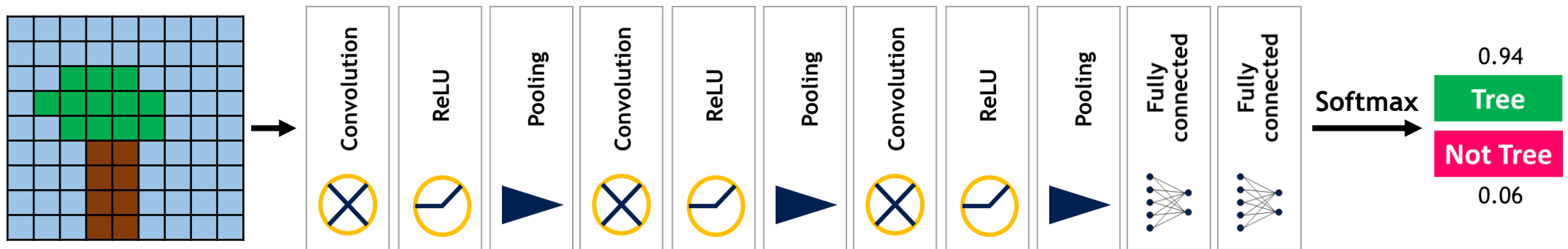
Putting everything together

- A set of pixels becomes a set of votes.



Learning from Images

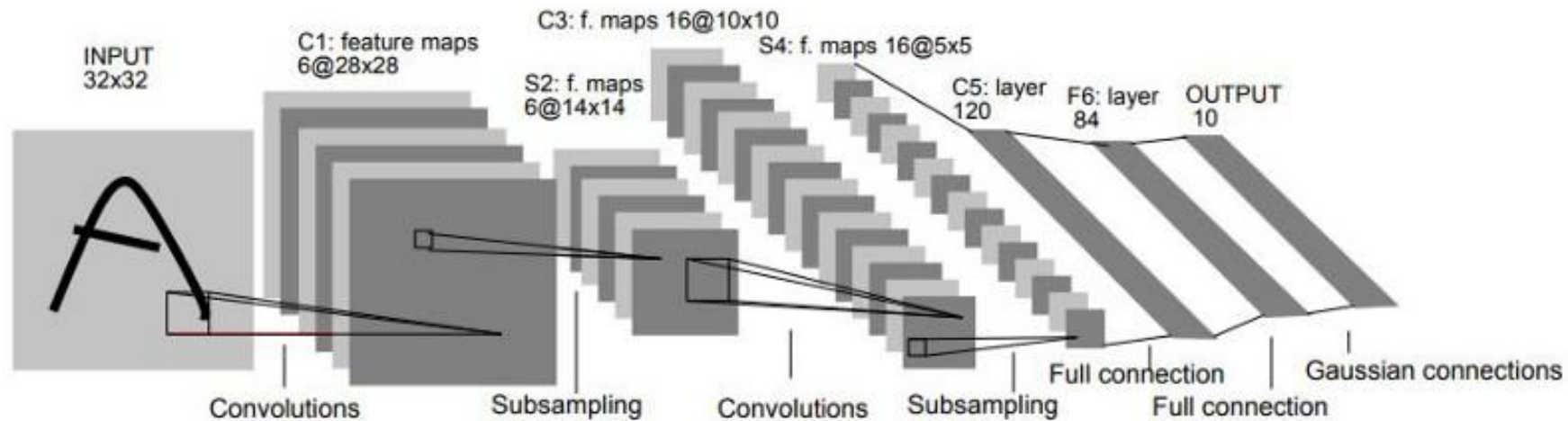
- **Convolution:** Apply filters to generate feature maps.
- **Non-linearity:** Often ReLU.
- **Pooling:** Downsampling operation on each feature map.
- **Training** model with image data.
- **Learn** weights of filters in convolutional layers using **backpropagation**.



Case Studies

LeNet (1998)

- 5 convolutional layers + average pooling
- Trained on MNIST digits
- First successful CNN for handwritten digit recognition



To learn more:



LeCun, Y., Bottou, L., Bengio, Y. and Haffner, P., 2002. *Gradient-based learning applied to document recognition*. Proceedings of the IEEE, 86(11), pp.2278-2324.

Image: LeCun et al., 1998

The ImageNet challenge

- Major computer vision benchmark
- Ran from 2010 to 2017
- 1.4M images, 1000 classes
- Image classification

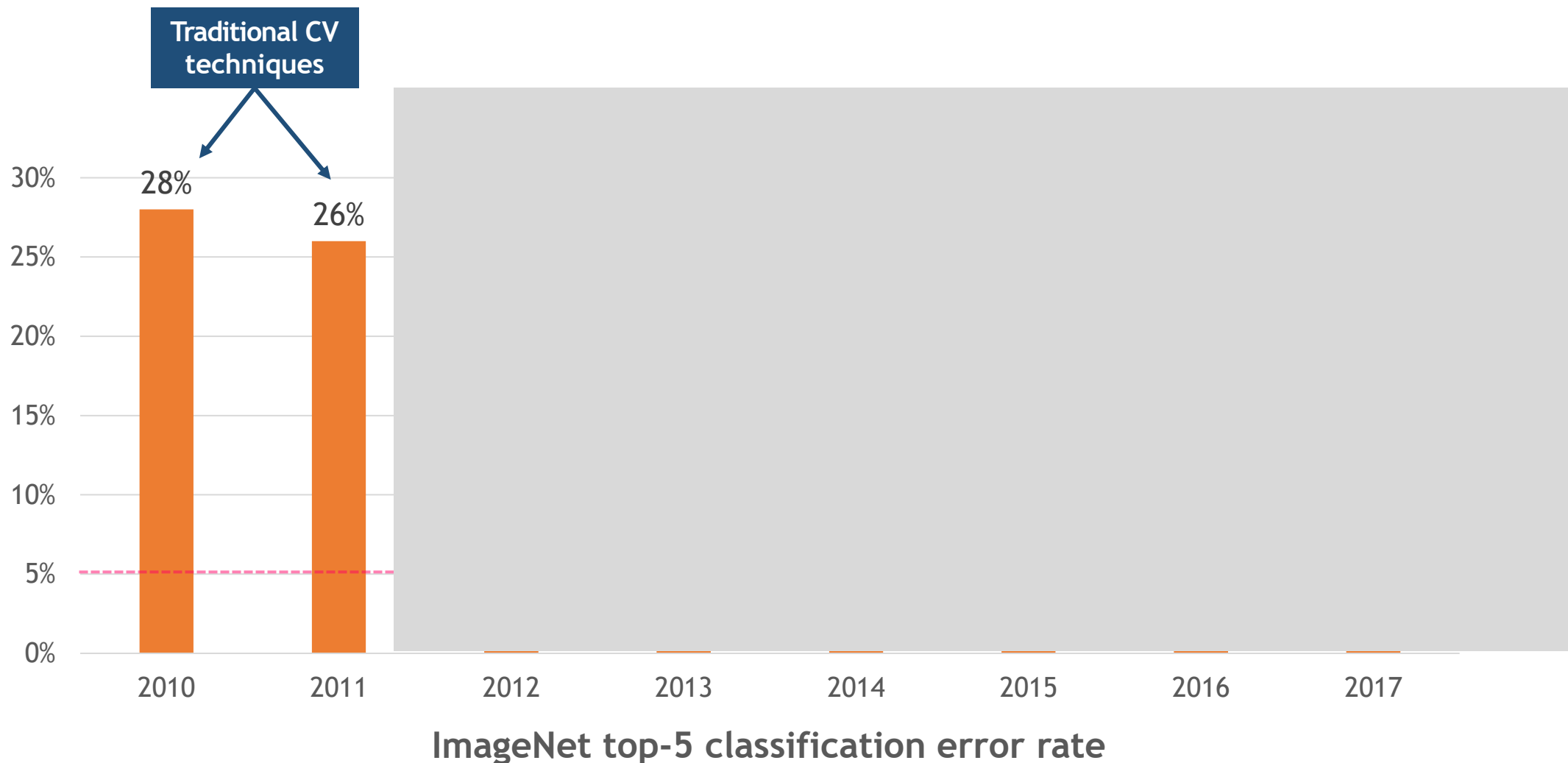


To learn more:

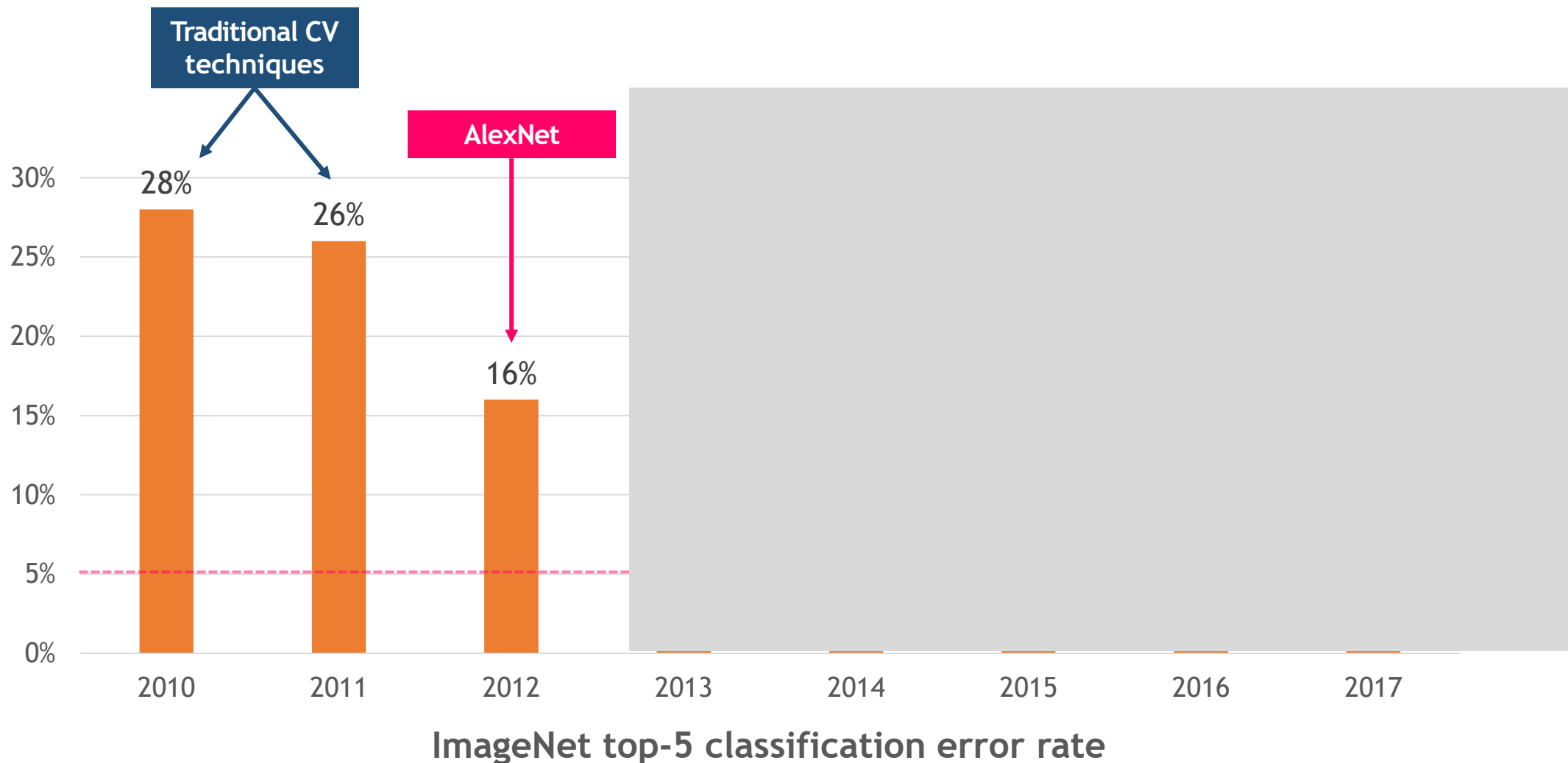


Russakovsky, Olga et al. *ImageNet Large Scale Visual Recognition Challenge* International Journal of Computer Vision 115.3 (2015)

The ImageNet challenge



The ImageNet challenge



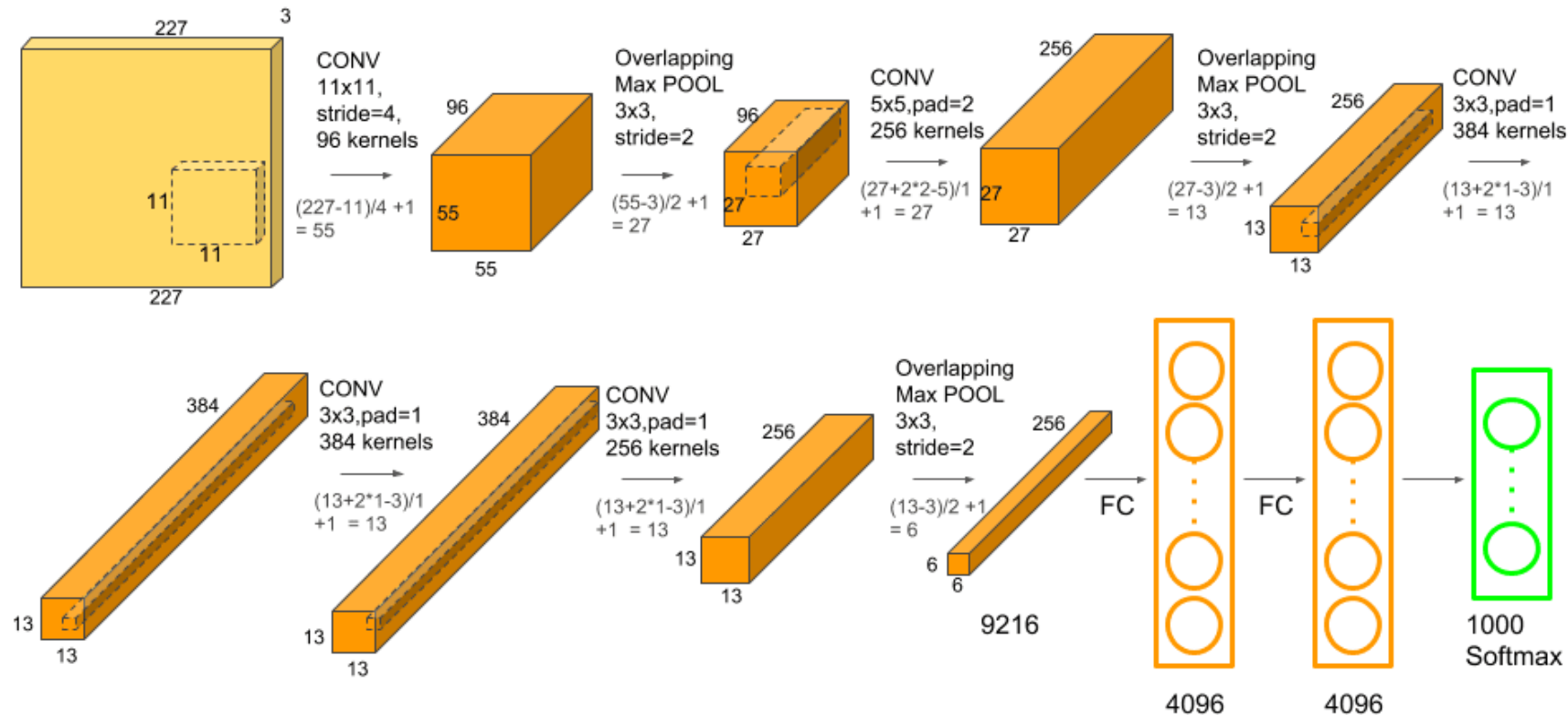
AlexNet (2012)

To learn more:



Krizhevsky, A.; Sutskever, I.; Hinton, G.E. *ImageNet classification with deep convolutional neural networks* Neural Information Processing Systems (2012)

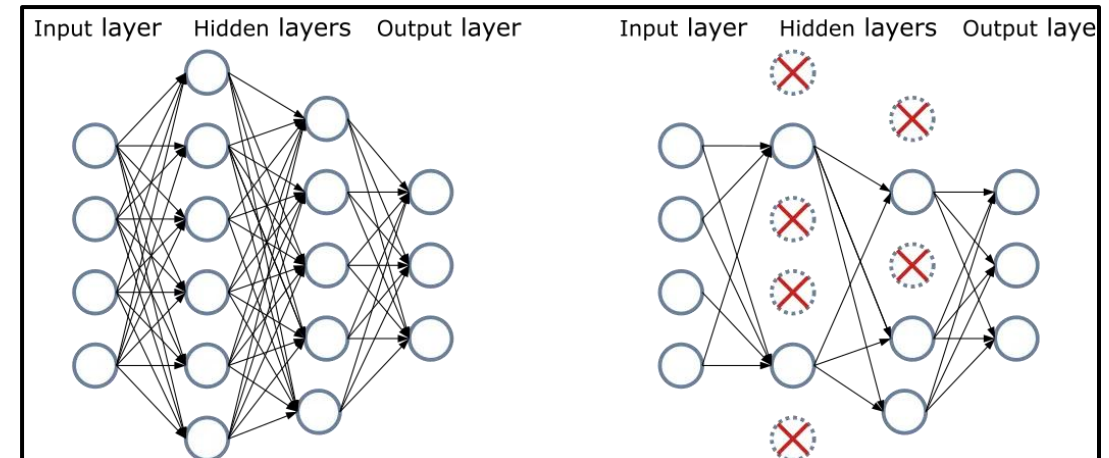
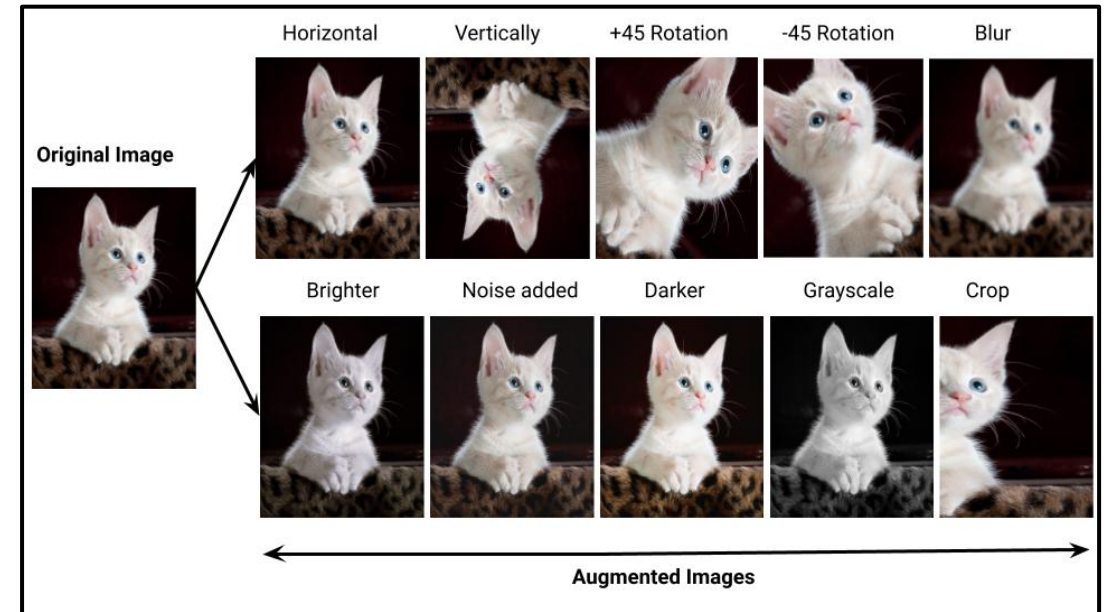
- Deep 8-layer CNN trained end-to-end on GPUs
- 5 conv + 3 FC layers



AlexNet (2012)

- Incorporated several techniques for training:

- ReLU
- GPU
- Data augmentation
- Dropout



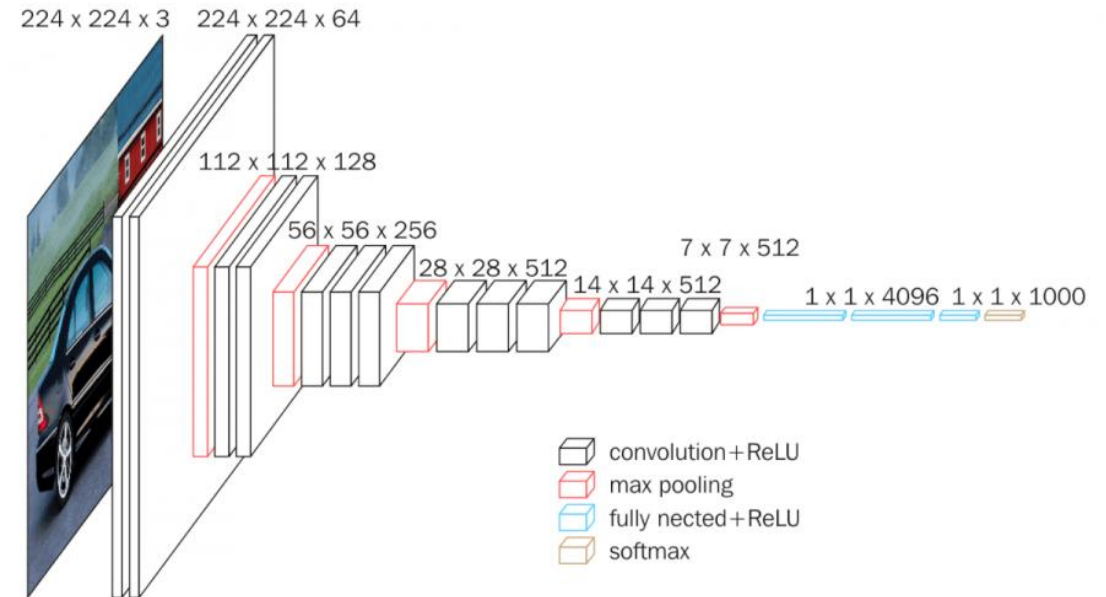
To learn more:



Krizhevsky, A.; Sutskever, I.; Hinton, G.E. *ImageNet classification with deep convolutional neural networks* Neural Information Processing Systems (2012)

VGGNet (2014)

- Use small, stacked 3×3 filters throughout
- 16-19 layers deep; every block = 2-3 conv + max-pool
- Demonstrated that deeper networks with small kernels outperform shallow, wide ones; inspired many later models

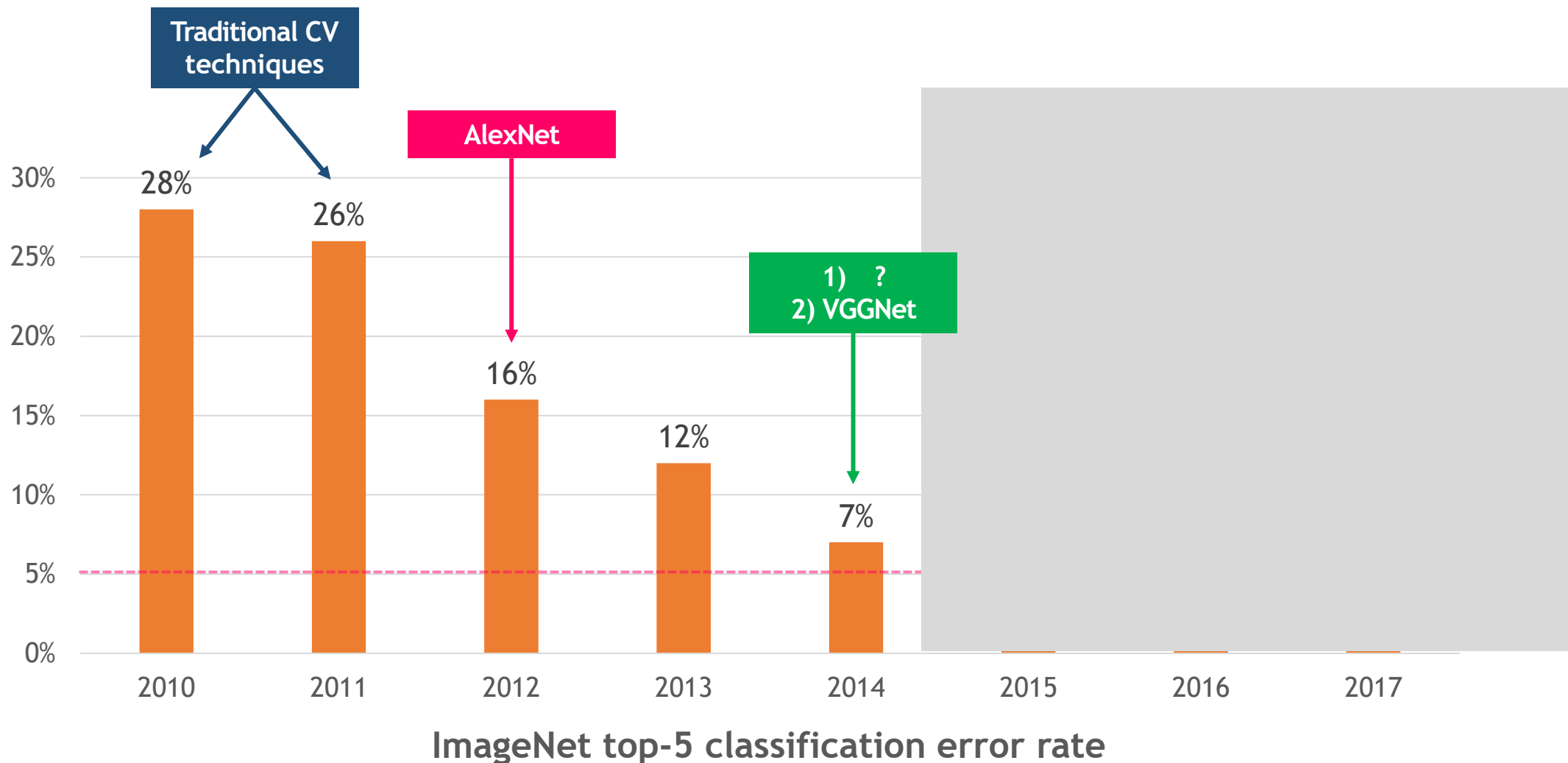


To learn more:

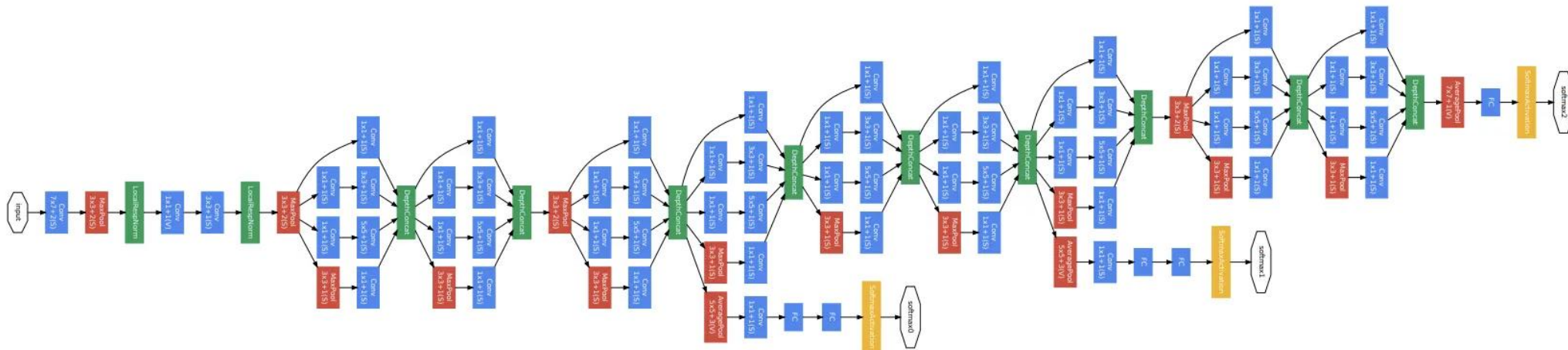


Krizhevsky, A.; Sutskever, I.; Hinton, G.E. *ImageNet classification with deep convolutional neural networks* Neural Information Processing Systems (2012)

The ImageNet challenge



GoogLeNet (2014)



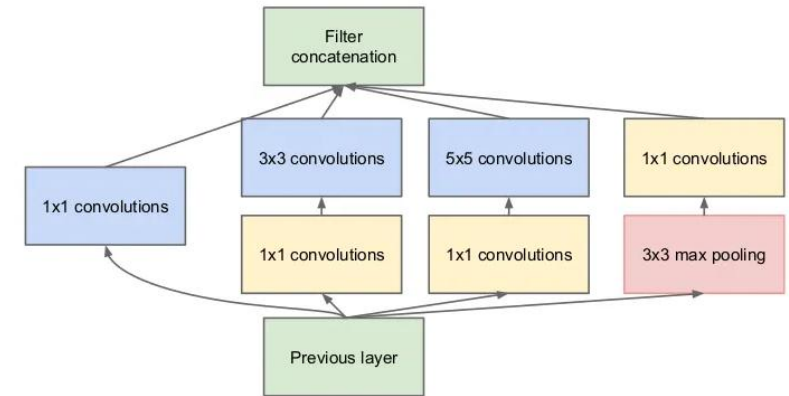
To learn more:

 Szegedy, C., Liu, W., Jia, Y., Sermanet, P., Reed, S., Anguelov, D., Erhan, D., Vanhoucke, V. and Rabinovich, A., 2015. *Going deeper with convolutions*. In Proceedings of the IEEE conference on computer vision and pattern recognition.

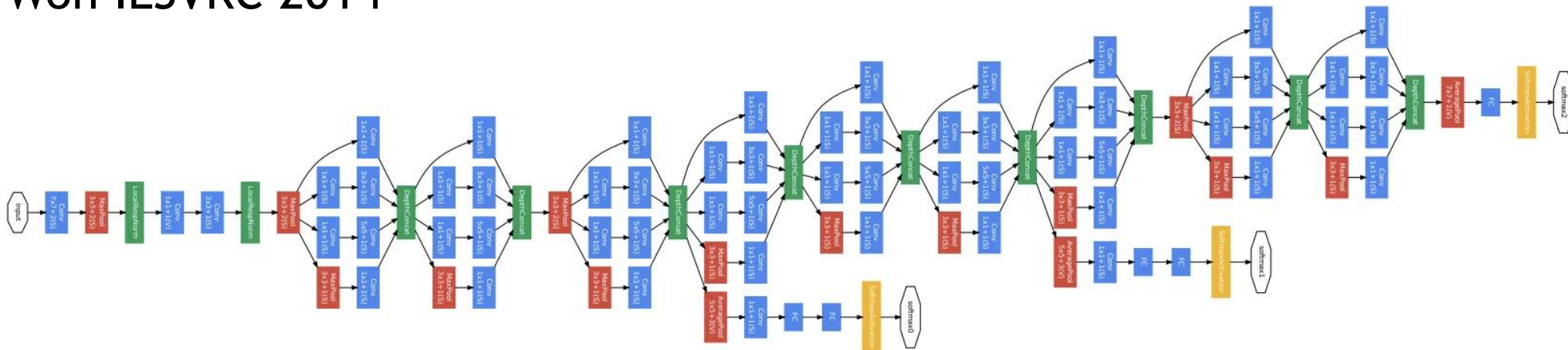
Images: Szegedy et al., 2015

GoogLeNet (2014)

- Deeper nets were becoming too heavy and computationally expensive
- Introduced **Inception modules**
- 22 layers deep but computationally lighter than VGG using 1×1 bottlenecks
- Won ILSVRC 2014



(b) Inception module with dimension reductions



To learn more:

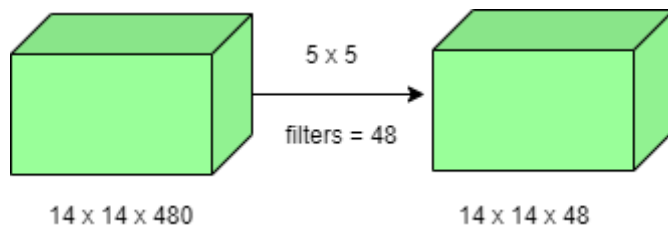


Szegedy, C., Liu, W., Jia, Y., Sermanet, P., Reed, S., Anguelov, D., Erhan, D., Vanhoucke, V. and Rabinovich, A., 2015. *Going deeper with convolutions*. In Proceedings of the IEEE conference on computer vision and pattern recognition.

Images: Szegedy et al., 2015

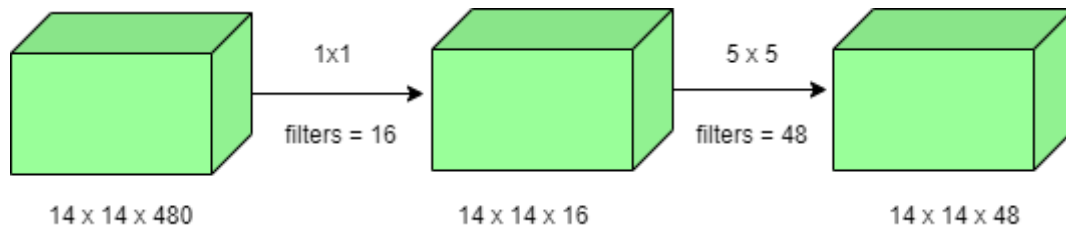
GoogLeNet (2014)

Example Comparison:



Without 1x1 Convolution (Direct):

Operations = (Output volume) × (Filter operations per position)
= $(14 \times 14 \times 48) \times (5 \times 5 \times 480) = 9,408 \times 12,000$
= **112.9M** operations



With 1x1 Convolution (Bottleneck):

Step 1 - Compress with 1x1:

$(14 \times 14 \times 16) \times (1 \times 1 \times 480) = 3,136 \times 480$
= 1.5M operations

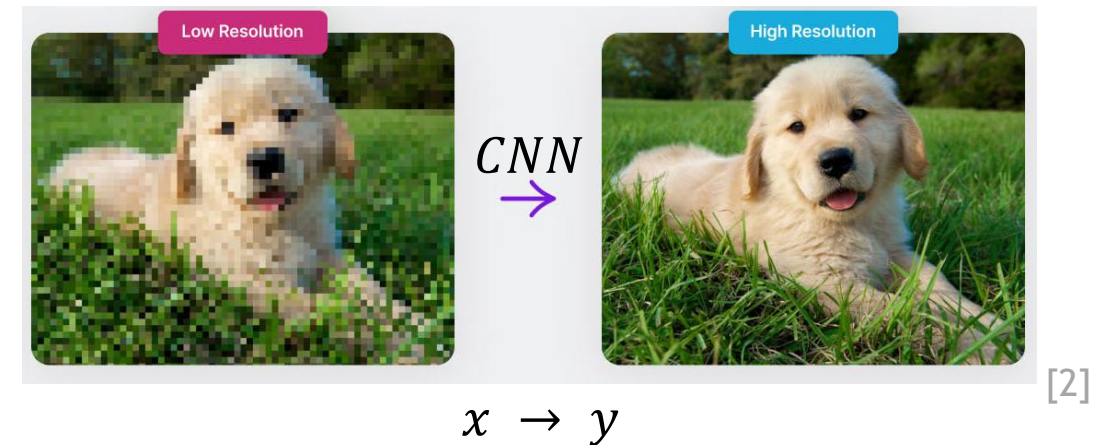
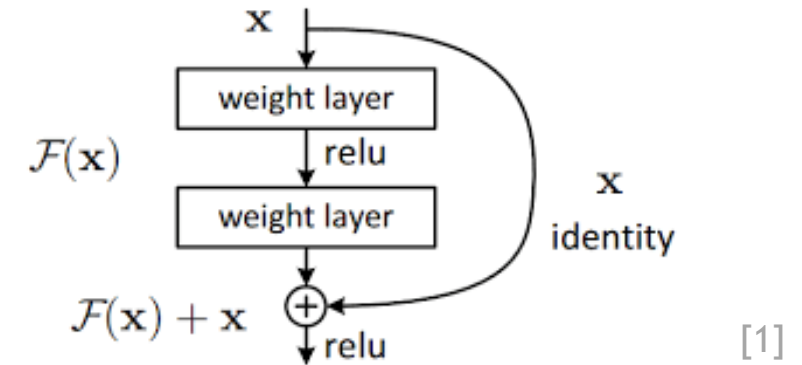
Step 2 - Apply 5x5 on compressed data:

$(14 \times 14 \times 48) \times (5 \times 5 \times 16) = 9,408 \times 400$
= 3.8M operations

Total: 1.5M + 3.8M = **5.3M** operations

ResNet (2015)

- **Skip connections:** Instead of learning a direct mapping $H(x)$, each layer learns a residual $F(x) = H(x) - x$. The output is $F(x) + x$.
 - helps training by addressing the vanishing gradient problem
- Original version: 152 layers
- Many variants, e.g., ResNet-18, ResNet-50



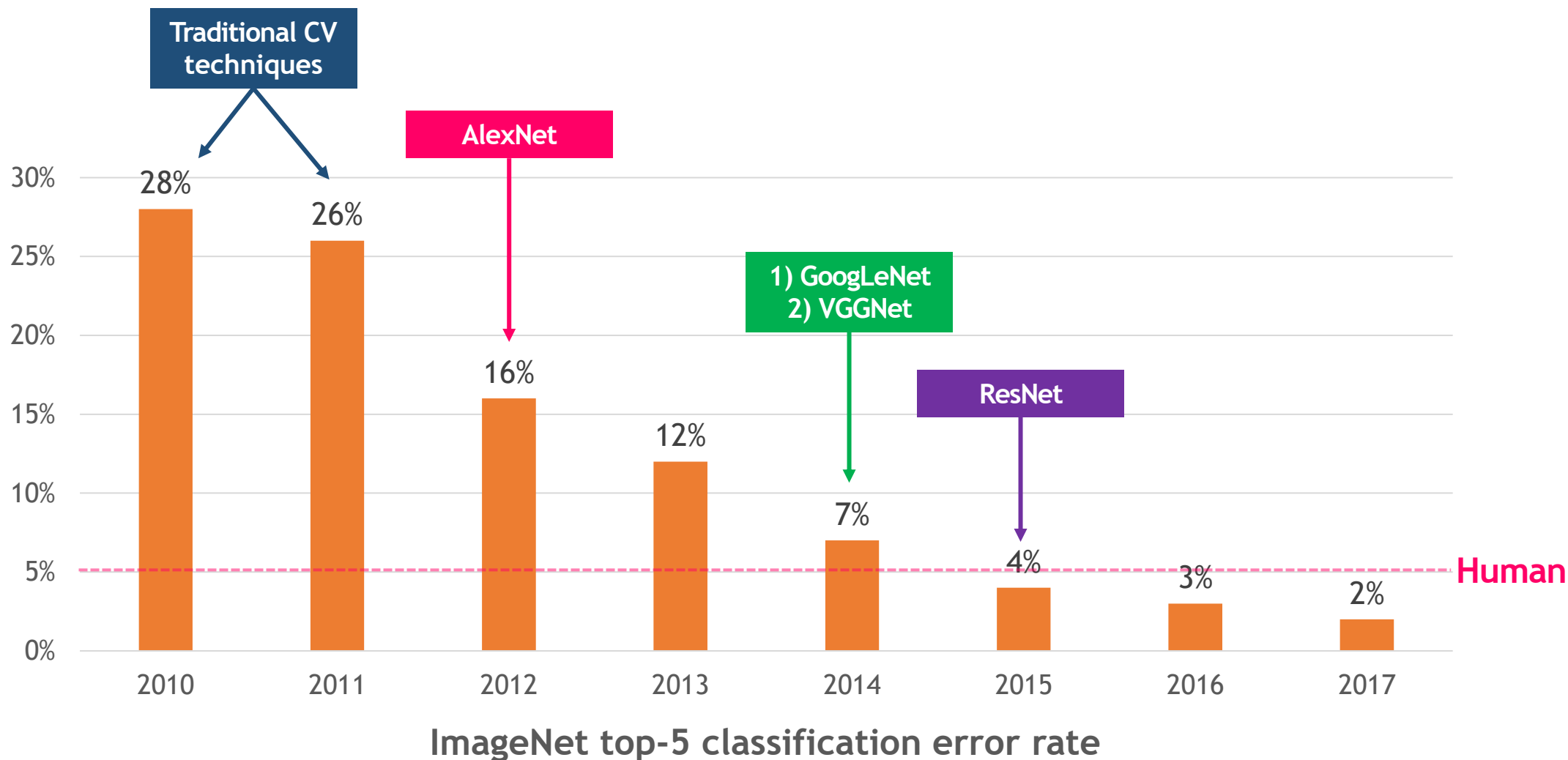
Intuition: Instead of learning the mapping from x to y , create a copy of x and only try to model *the difference between x and the desired output y .*



He, K., Zhang, X., Ren, S. and Sun, J., 2016. *Deep residual learning for image recognition*. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 770-778).

Images: [1] He et al., 2016
[2] <https://www.v7labs.com/>

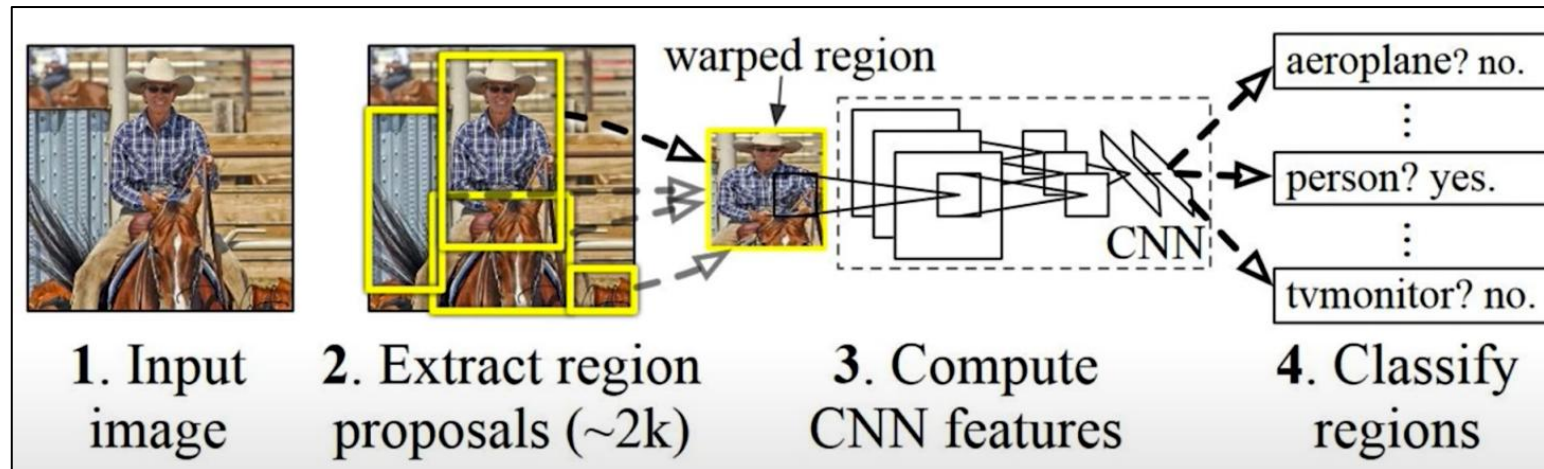
The ImageNet challenge



Applications

Object detection

- **R-CNN algorithm:** Find regions that we think have objects. Use CNN to classify.



To learn more:

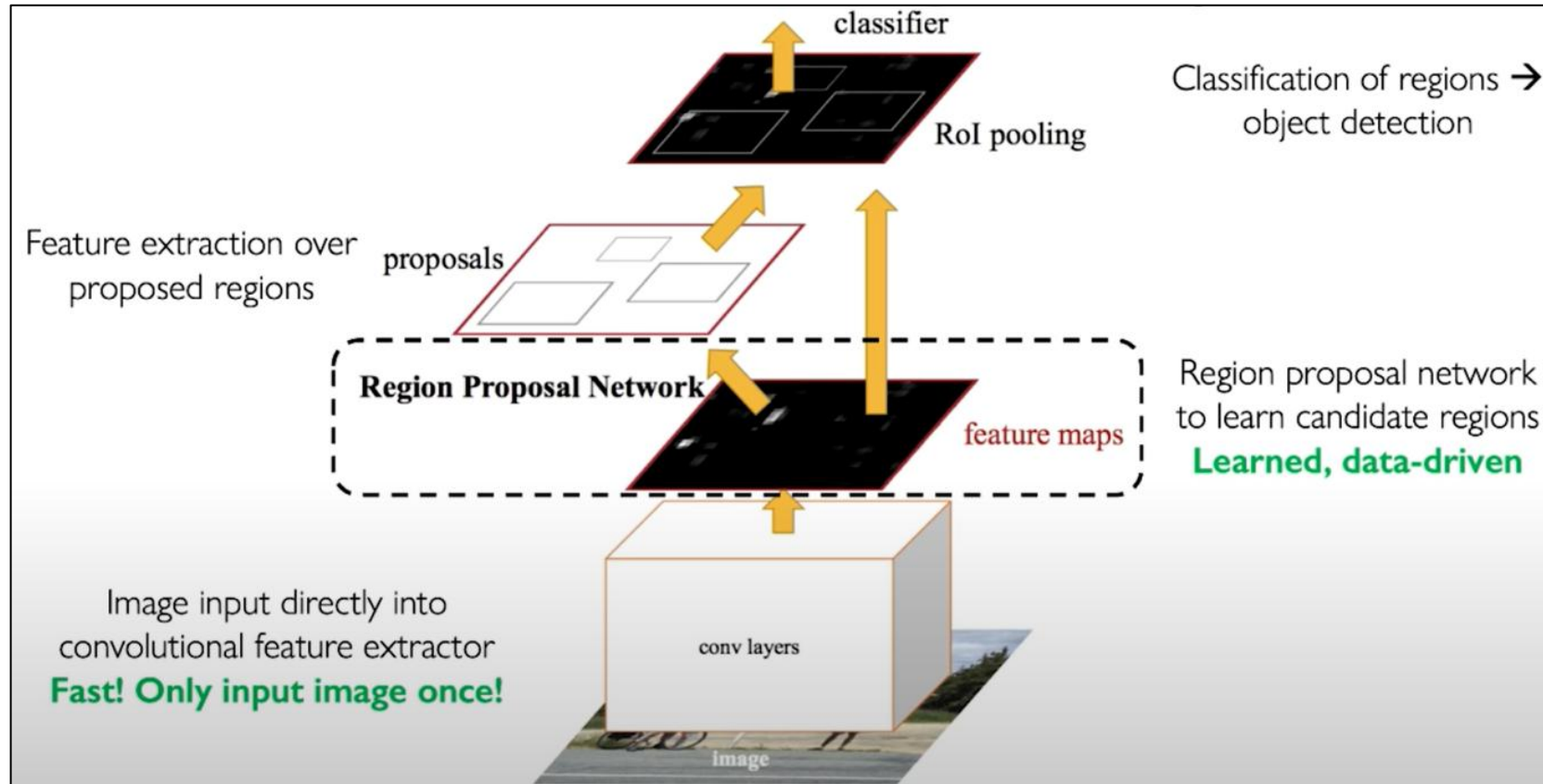


Girshick, R., 2015. **Fast r-cnn**. In Proceedings of the IEEE international conference on computer vision (pp. 1440-1448).

Image: Amini, *Convolutional Neural Networks*.

Object detection

- **Faster R-CNN:** Learns region proposals



To learn more:

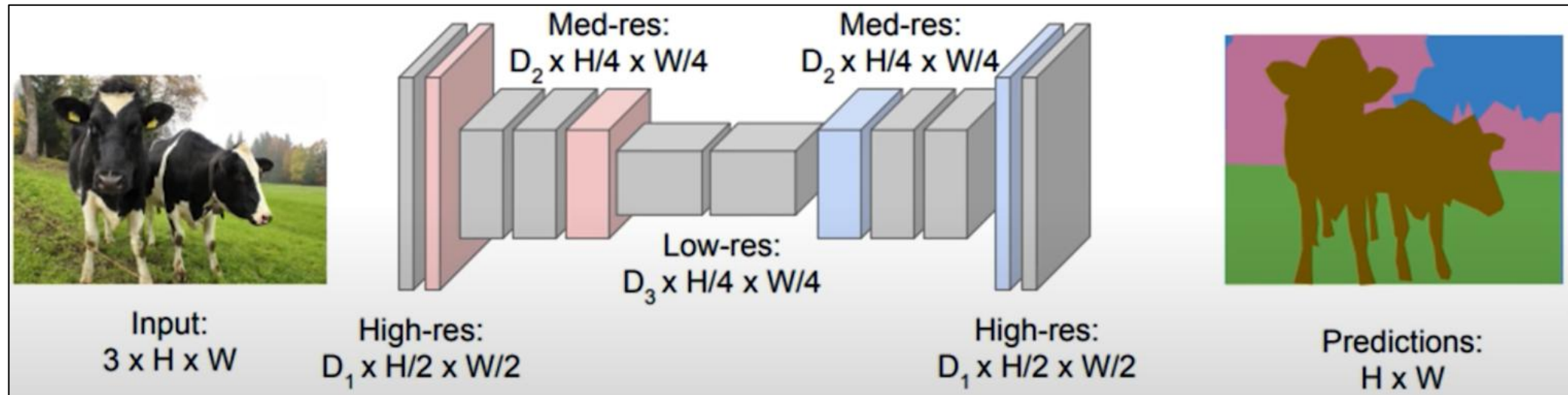


Ren, S., He, K., Girshick, R. and Sun, J., 2016. ***Faster R-CNN: Towards real-time object detection with region proposal networks.*** IEEE transactions on pattern analysis and machine intelligence.

Image: Amini, *Convolutional Neural Networks*.

Semantic Segmentation

- Fully Convolutional Networks



To learn more:

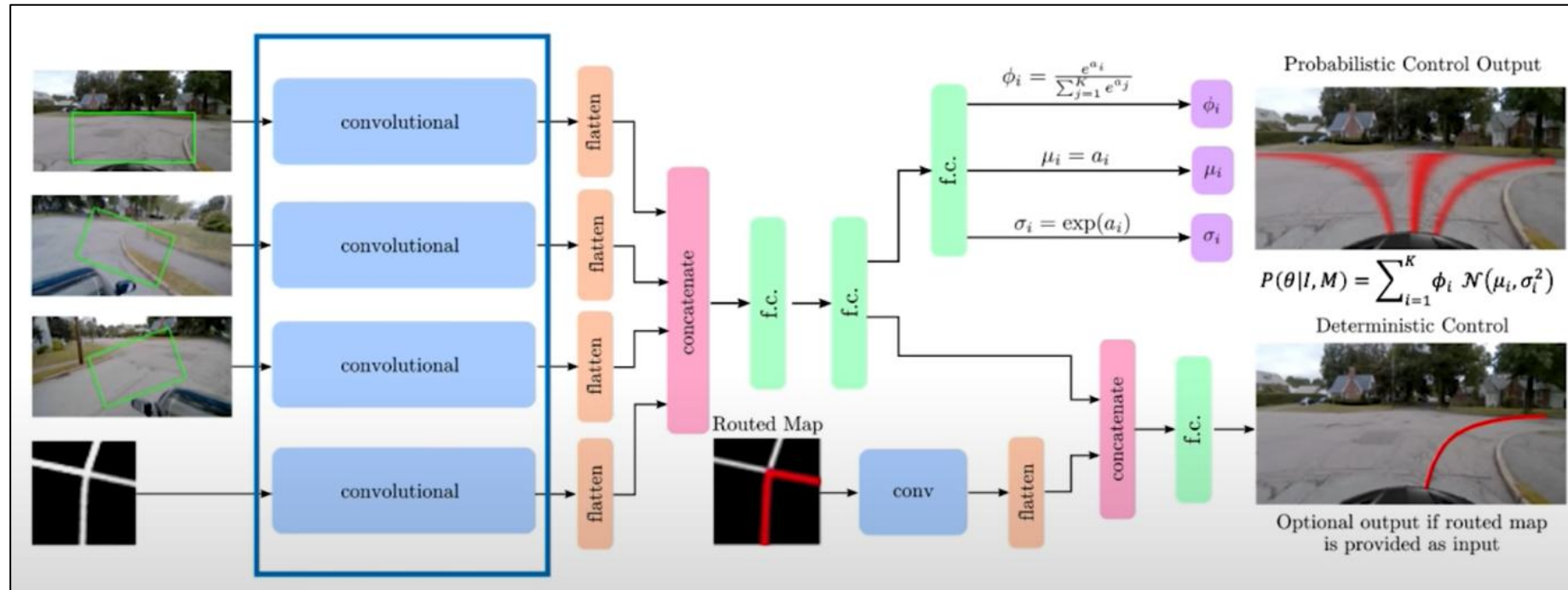


Long, J., Shelhamer, E. and Darrell, T., 2015. **Fully convolutional networks for semantic segmentation**. In Proceedings of the IEEE conference on computer vision and pattern recognition.

Image: Amini, *Convolutional Neural Networks*.

Continuous Control: Autonomous Navigation from Vision

- Entire model is trained end-to-end without any human labelling



To learn more:



Amini, A., Rosman, G., Karaman, S. and Rus, D., 2019, May. **Variational end-to-end navigation and localization.** In 2019 International Conference on Robotics and Automation (ICRA). IEEE.

Image: Amini, *Convolutional Neural Networks*.

One architecture, many applications

